

Appunti di Multimedia

Giacomo Simonetto

Secondo semestre 2025-26

Sommario

Appunti del corso di Multimedia (Reti di Calcolatori) della facoltà di Ingegneria Informatica dell'Università di Padova.

Indice

1	Segnali e servizi multimediali	4
1.1	Segnali multimediali	4
1.2	Servizi multimediali	4
1.3	Requisiti e obiettivi dei servizi multimediali	5
2	Trasmissione dell'informazione multimediale	6
2.1	Fasi di trasmissione dell'informazione	6
2.2	Campionamento	7
2.3	Quantizzazione	8
2.4	Inverse bit mapper (IBM) e parallel-to-serial converter (P/S)	9
2.5	Serial-to-parallel converter (S/P) e bit mapper (BM)	9
2.6	Interpolazione	9
3	Rappresentazione dei segnali multimediali	11
3.1	Rappresentazione di segnali audio	11
3.2	Rappresentazione di immagini	12
3.3	Rappresentazione di video	13
4	Codifica vocale	14
4.1	Caratteristiche della voce e classi di codificatori	14
4.2	Codificatore vocoder basato su LPC-10	14
4.3	Codificatori più avanzati - ACELP, AMR-WB e Opus	15
4.4	Codificatore Opus - RFC 6716	16
5	Codifica lossless	17
5.1	Introduzione alle codifiche lossless	17
5.2	Codifica di Huffman	18
5.3	Codifica aritmetica	19
5.4	Codifiche basate su dizionari - Lempel-Ziv, DEFLATE e ASN	19
5.5	Codifica Exp-Golomb	20
5.6	Codifica per categoria e ampiezza	20
5.7	Codifica lossless predittiva	21
5.8	Codifica lossless tramite reti neurali	21
6	Codifica JPEG	22
6.1	Processo di codifica JPEG	22
6.2	Discrete Cosine Transform - DCT	22
6.3	Quantizzazione dei coefficienti DCT	23
6.4	Codifica lossless dei coefficienti DCT	24
6.5	Sintassi del formato JPEG baseline (1992)	25
6.6	Evoluzione dei formati di compressione delle immagini	25
7	Codifica video	26
7.1	Principi di compressione video	26
7.2	Stima e compensazione del movimento per block matching	26
7.3	Tipi di immagine e GOP	28
7.4	Codificatori e decodificatori ibridi	28
7.5	Standard di codifica video principali	30
8	Metriche di rete	31
8.1	Architettura di rete a strati	31
8.2	Paradigma di trasmissione a pacchetto	32
8.3	Metriche di traffico	32
8.4	Metriche di ritardo	33
8.5	Prodotto banda-ritardo (BDP) e ritrasmissione dei pacchetti	34
8.6	Metriche di affidabilità	35

9	Streaming	37
9.1	Protocolli real-time (push) e streaming (pull)	37
9.2	Adattamento del tasso di codifica video	37
9.3	Dynamic adaptive streaming over HTTP - DASH	38

1 Segnali e servizi multimediali

1.1 Segnali multimediali

Medium e multimedia

- un medium (o media al plurale) è un mezzo per rappresentare, immagazzinare e trasportare informazioni, esempi di media sono testo, suono, immagini, video, ecc.
- un multimedia è un sistema di rappresentazione/immagazzinamento/trasporto di informazioni che utilizza più media diversi

Segnali e comunicazioni multimediali

- un segnale è una funzione di una o più variabili dipendenti in funzione di una o più variabili indipendenti (tempo per audio, spazio per immagini, spazio-tempo per video, ecc.), può essere analogico (dominio e immagine continui) o digitale (dominio e immagine discreti) e dal punto di vista ingegneristico è un mezzo che trasporta informazioni
- un segnale multimediale è la rappresentazione di uno o più media (audio, video, immagini, ecc.) come segnali digitali (discreti e quantizzati) che trasmettono informazioni
- una comunicazione multimediale è la trasmissione di segnali multimediali

Digitalizzazione dei segnali analogici e multimediali

Per rappresentare delle informazioni provenienti da segnali analogici (es. suono, immagini) in segnali multimediali è necessario eseguire le operazioni di campionamento, quantizzazione e codifica.

Il processo di digitalizzazione comporta inevitabilmente una perdita di informazione. In quelli attualmente utilizzati, però, non è possibile percepire tale perdita. Per questo motivo i segnali multimediali digitali saranno considerati “segnali originali” in quanto indistinguibili da quelli analogici di partenza.

Sorgenti statiche e dinamiche

- una sorgente statica produce informazioni tempo invarianti (es. immagini, testi), supportano bene jitter, latenze e variazioni di bitrate, ma non supportano bene errori di trasmissione
- una sorgente dinamica produce informazioni in funzione del tempo (es. audio, video), richiedono sincronismo tra diversi media (es. audio, video e testo dei sottotitoli), supportano bene errori di trasmissione, ma sono sensibili a jitter e latenza

1.2 Servizi multimediali

Categorie dei servizi multimediali

Un servizio multimediale è un sistema di rappresentazione, immagazzinamento e trasporto di informazioni basato su segnali multimediali. I servizi multimediali possono essere classificati in streaming e bulk transfer, in base a come i dati trasmessi vengono fruiti dall'utente finale.

Streaming

Un servizio streaming permette la fruizione dei media anche prima della completa ricezione dei dati. È composto da una sorgente dinamica che invia i dati a passo costante (stream) a uno o più ricevitori che li memorizzano in un buffer (playout) per poi riprodurli. Richiede una attenta gestione del buffer, della banda, della latenza, del jitter e dell'ordine con cui i pacchetti vengono ricevuti. In base al tipo di fruizione dei media, lo streaming può essere:

- **stored**: il contenuto viene trasmesso in maniera asincrona, dopo essere stato prodotto, elaborato e memorizzato in una memoria dedicata (server); es. video on demand come netflix e youtube, podcast e musica come spotify, ecc.
- **live**: il contenuto viene trasmesso appena viene prodotto, con latenze di poche decine di secondi; es. eventi sportivi in diretta, ecc.

- **real-time**: caso particolare dei servizi live streaming in cui la latenza non deve superare i 150 ms; es. videochiamate, teleconferenze, giochi online, ecc.

I servizi di streaming in cui l'utente può interagire con il contenuto multimediale sono detti interattivi.

- **streaming interattivo**: l'utente può interagire con il contenuto multimediale, ad esempio attraverso chat, votazioni, commenti, domande in trasmissioni live o mettere in pausa, riavvolgere o saltare parti del contenuto in servizi stored
- **streaming non interattivo**: l'utente non può interagire con il contenuto multimediale, ad esempio nei servizi real time

Bulk transfer

Un servizio bulk transfer permette la fruizione dei media solo dopo la completa ricezione dei dati. Sono costituiti principalmente da sorgenti statiche, per cui tollerano bene latenze e jitter, ma non tollerano errori di trasmissione. Come per i servizi streaming, anche i servizi bulk transfer sono classificati in:

- **stored**: il contenuto viene trasmesso in maniera asincrona, es. pagine web, download di file, ecc.
- **live**: il contenuto viene trasmesso appena viene prodotto, es. messaggistica, email, ecc.
- non esistono servizi bulk transfer real-time

1.3 Requisiti e obiettivi dei servizi multimediali

Parametri di misurazione dei requisiti

- banda/tasso/throughput: quantità di informazione trasmessa per unità di tempo, in bit/s
- latenza: ritardo dovuto alla trasmissione dei dati, in secondi
- jitter: variazione della latenza, in secondi
- tasso d'errore: percentuale di bit errati rispetto al totale dei bit trasmessi, in %
- complessità: costo computazionale, consumo, ritardi di computazione, ...
- fedeltà al segnale originale: compromesso tra qualità e tasso d'errore

Obiettivo dei servizi multimediali

L'obiettivo dei servizi multimediali è quello di utilizzare un'infrastruttura di rete già presente (es. rete internet) e sviluppare opportuni protocolli per privilegiare determinate esigenze (banda, latenza) rispetto ad altre, per garantire la migliore qualità possibile in funzione dei limiti strutturali imposti dalla rete.

Classificazione banda-latenza

- bassa banda e latenza: controllo remoto
- alta banda, bassa latenza: AR/VR, guida autonoma
- bassa banda, alta latenza: IoT, sensori
- alta banda, alta latenza: streaming video

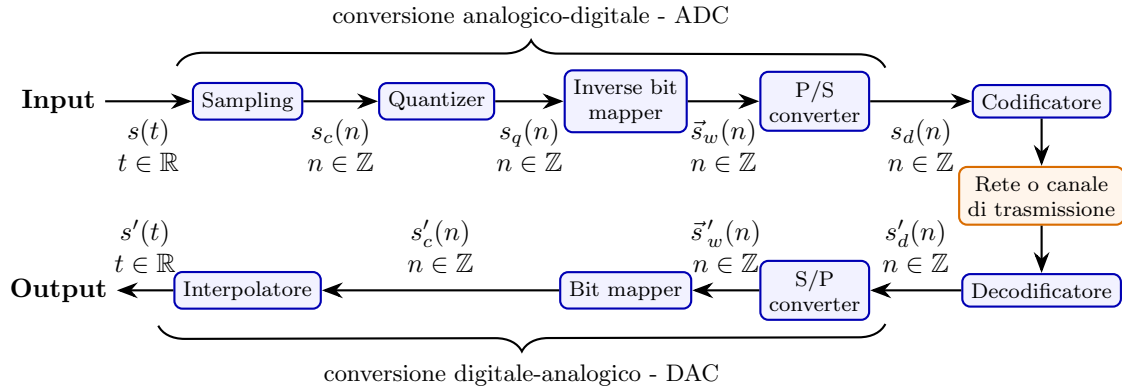
Servizi URLL - Ultra Reliable Low Latency

Esistono alcuni servizi che richiedono latenza e tasso d'errore estremamente bassi come ad esempio:

- automazione industriale, controllo del movimento (latenza ≤ 1 ms, tasso d'errore $\leq 10^{-9}$)
- tactile internet (latenza ≤ 1 ms, tasso d'errore $\leq 10^{-4}$)
- smart grid, controllo di sistemi energetici (latenza $\leq 3 - 5$ ms, tasso d'errore $\leq 10^{-7}$)
- intelligent transport system, comunicazione tra veicoli (latenza ≤ 5 ms, tasso d'errore $\leq 10^{-4}$)
- guida autonoma (latenza $\leq 15 - 20$ ms, tasso d'errore $\leq 10^{-4}$)
- controllo remoto (latenza $\leq 5 - 100$ ms, tasso d'errore $\leq 10^{-2}$)
- automazione processi industriali, controllo di macchinari (latenza ≤ 100 ms, tasso d'errore $\leq 10^{-5}$)

2 Trasmissione dell'informazione multimediale

2.1 Fasi di trasmissione dell'informazione



Il processo di trasmissione dell'informazione di segnali analogici attraverso servizi multimediali digitali prevede le seguenti fasi:

1. conversione analogico-digitale (ADC): conversione dei segnali analogici in segnali digitali, comprende le operazioni di campionamento, quantizzazione, e conversione dei simboli in sequenze seriali di bit
2. codifica (COD): compressione delle sequenze di bit per ridurre la quantità di informazione da trasmettere e aggiunta di ridondanza per la protezione dai possibili errori di trasmissione
3. trasmissione (NET): invio dei simboli attraverso un canale di comunicazione della rete presente
4. decodifica (DEC): conversione dei simboli ricevuti in segnali digitali
5. conversione digitale-analogico (DAC): ricostruzione dei segnali analogici a partire dai segnali digitali ricevuti, comprende la conversione dei simboli in sequenze seriali di bit e una fase di interpolazione

Domini dei segnali analogici e digitali

Un **segnale analogico** è una funzione di una o più variabili dipendenti continue in funzione di altre variabili indipendenti continue.

$$f : \mathcal{X} \subseteq \mathbb{R}^m \rightarrow \mathcal{Y} \subseteq \mathbb{R}^n \quad f(\mathcal{X}, \mathcal{Y}) \in \mathbb{R}^m \times \mathbb{R}^n$$

- segnale audio analogico su nastro magnetico: $f(t) \in \mathbb{R} \times \mathbb{R}$
- segnale immagine analogico su diapositiva: $f(x, y) \in \mathbb{R}^2 \times \mathbb{R}$
- segnale video analogico, non campionato: $f(x, y, t) \in \mathbb{R}^3 \times \mathbb{R}$

Un **segnale analogico campionato** è un segnale analogico in cui le variabili indipendenti sono state discretizzate attraverso un'acquisizione a intervalli regolari dei valori delle variabili dipendenti. Tale processo di acquisizione è detto campionamento (approfondito in seguito).

$$f : \mathcal{X} \subseteq \mathbb{Z}^m \rightarrow \mathcal{Y} \subseteq \mathbb{R}^n \quad f(\mathcal{X}, \mathcal{Y}) \in \mathbb{Z}^m \times \mathbb{R}^n$$

- segnale audio campionato: $f(k) \in \mathbb{Z} \times \mathbb{R}$
- segnale immagine campionato: $f(m, n) \in \mathbb{Z}^2 \times \mathbb{R}$
- segnale video campionato: $f(m, n, k) \in \mathbb{Z}^3 \times \mathbb{R}$

Un **segnale digitale** è una funzione di una o più variabili dipendenti discrete in funzione di altre variabili indipendenti discrete. Si ottiene assegnando un valore discreto alle variabili dipendenti del segnale analogico campionato attraverso un processo di quantizzazione (approfondito in seguito).

$$f : \mathcal{X} \subseteq \mathbb{Z}^m \rightarrow \mathcal{Y} \subseteq \mathbb{Z}^n \quad f(\mathcal{X}, \mathcal{Y}) \in \mathbb{Z}^m \times \mathbb{Z}^n$$

- segnale audio digitale su CD: $f(n) \in \mathbb{Z} \times \mathbb{Z}$
- segnale immagine digitale su monitor: $f(m, n) \in \mathbb{Z}^2 \times \mathbb{Z}$
- segnale video digitale su DVD: $f(m, n, k) \in \mathbb{Z}^3 \times \mathbb{Z}$

2.2 Campionamento

Trasformata di Fourier

Dato un segnale a tempo continuo $s(t)$ (con durata e ampiezza finita), la sua trasformata di Fourier restituisce lo spettro $S(f)$ che rappresenta il contenuto frequenziale del segnale $s(t)$.

- la trasformata di Fourier di un tono puro sinusoidale $s(t) = A \cdot \sin(2\pi f_0 t)$ è un impulso a frequenza f_0 con area costante
- la trasformata di Fourier del segnale $\text{sinc}(t)$ è un segnale $\text{rect}(f)$, il sinc viene detto segnale a banda limitata, proprio perché la sua trasformata di Fourier è limitata a un intervallo di frequenze $[f_m, f_M]$

Teorema del campionamento di Shannon e criterio di Nyquist

Un segnale $s(t)$ può essere ricostruito con errore nullo a partire dalla successione dei campioni $s_c(n)$ ottenuti dal suo campionamento se e solo se tale segnale è banda limitata con frequenza massima f_M e la frequenza di campionamento F_c è almeno il doppio della frequenza massima f_M del segnale $s(t)$.

$$s_c(n) = s(t)|_{t=n \cdot T_c} \quad \text{con } F_c = 1/T_c \geq 2f_M$$

La relazione $F_c \geq 2f_M$ è detta criterio di Nyquist.

Formula di interpolazione ideale

Se il criterio di Nyquist è soddisfatto, il segnale $s(t)$ può essere ricostruito a partire dai campioni $s_c(n)$ attraverso la formula di interpolazione ideale (convoluzione tra segnale campionato e segnale $\text{sinc}(t)$):

$$s(t) = \sum_{n \in \mathbb{Z}} s_c(n) \cdot \text{sinc}\left(\frac{t - nT_c}{T_c}\right)$$

La formula di interpolazione ideale consiste nel sommare tanti segnali $\text{sinc}(t)$ centrati negli istanti di campionamento nT_c , scalati orizzontalmente per occupare tutto il periodo di campionamento $[-T_c, T_c]$ e pesati dai valori dei campioni ai rispettivi istanti $s_c(n)$.

Filtraggio di segnali limitati nel tempo con banda illimitata

I segnali multimediali reali hanno durata finita e di conseguenza sono a banda illimitata. Non è possibile, quindi, applicare strettamente il teorema di Shannon-Nyquist, di conseguenza il segnale campionato non può essere ricostruito con errore nullo.

Si nota, però, che la maggior parte dei segnali si concentra solo su una banda limitata di frequenze. Di conseguenza, è possibile applicare un filtro passa-basso con frequenza di taglio $f_c/2$ senza perdere informazioni rilevanti del segnale originale e senza introdurre errori di distorsione percepibili. In questo modo si limita il contenuto frequenziale del segnale e risulta possibile applicare il teorema di Shannon-Nyquist garantendo una ricostruzione del segnale campionato con errore trascurabile.

Aliasing - frequenze spurie e sovrapposizione delle repliche dello spettro

Se si campiona un segnale che non rispetta il criterio di Nyquist, si verifica un fenomeno di aliasing, che consiste nello spostamento verso il basso delle alte frequenze del segnale campionato che non possono essere ricostruite correttamente. In altre parole nel segnale ricostruito sono presenti nuove frequenze spurie che non erano presenti nel segnale originale.

Dal punto di vista della trasformata di Fourier, il campionamento di un segnale $s(t)$ equivale alla replica in frequenza del suo spettro $S(f)$ compreso nel range $[-f_M, f_M]$ a intervalli regolari di F_c . Il fenomeno di aliasing corrisponde alla sovrapposizione (overlapping) di tali repliche se $f_M > F_c/2$.

Per limitare l'aliasing, si utilizza per l'appunto un filtro passa-basso sul segnale originale prima del campionamento per ridurre le alte frequenze che generano frequenze spurie o in altre parole per ridurre la banda ed evitare l'overlapping delle repliche.

2.3 Quantizzazione

Quantizzazione per livelli

Il processo di quantizzazione consiste nel discretizzare l'ampiezza di un segnale (si suppone già campionato) in un numero finito di L livelli. Si definisce, inoltre, la quantità di informazione o costo di codifica $R = \log_2 L$ come il numero di bit necessari a rappresentare ogni livello.

Aumentando il numero di livelli L aumenta la fedeltà del segnale quantizzato rispetto al segnale originale, ma aumenta anche il costo di codifica R per rappresentare ogni campione quantizzato.

Quantizzazione uniforme

In un quantizzatore uniforme si suddivide l'intervallo di ampiezza del segnale in L livelli uniformemente distribuiti, ciascuno con un passo di quantizzazione Δ costante. Il processo di quantizzazione consiste nel rappresentare ogni campione con il più vicino livello di quantizzazione (o multiplo di Δ).

$$Q(x, \Delta) = \Delta \cdot \text{round}\left(\frac{x}{\Delta}\right) \quad \text{con } x \in [-A, A], \quad L = 2A/\Delta, \quad \varepsilon = \Delta/2$$

Quantizzatori mid-rise, mid-tread e dead zone

In base a come viene trattato il valore zero, i quantizzatori uniformi vengono classificati in:

- **mid-rise**: il valore zero è un valore di soglia tra due livelli di quantizzazione, se il segnale presenta piccoli valori di rumore attorno allo zero, questi vengono amplificati e degenerano in uno dei due livelli adiacenti
- **mid-tread**: il valore zero è associato ad un livello di quantizzazione centrato su zero e passo di quantizzazione Δ uguale a quello degli altri livelli, se il segnale presenta piccoli valori di rumore attorno allo zero, questi vengono eliminati e il segnale quantizzato viene riportato a zero
- **dead zone**: come il mid-rise, ma con un passo di quantizzazione maggiore rispetto agli altri livelli, serve per comprimere ulteriormente i dati sparsi (con tanti valori piccoli non significativi)

Rumore di quantizzazione e distorsione

Il processo di quantizzazione porta inevitabilmente ad una perdita di informazione in quanto è un'operazione irreversibile. Inoltre si osserva che la qualità del segnale degrada molto velocemente al diminuire del numero di livelli L . Per questo motivo si utilizzano Δ piccoli e L elevati per garantire una buona fedeltà al segnale originale e non introdurre distorsioni o artefatti percepibili. Di solito si utilizzano quantizzatori a 8 bit per canale di colore (RGB) e 16 bit per campionamento audio.

La quantizzazione può essere utilizzata una seconda volta come tecnica di compressione all'interno dei processi di codifica e può introdurre distorsioni evidenti allo scopo di diminuire la dimensione dei dati. La prima quantizzazione della conversione DAC, però, non deve indurre distorsioni percepibili.

Costo della quantizzazione

Il costo della quantizzazione è dato dal numero di bit necessari a rappresentare ogni campione quantizzato, ovvero al tasso di codifica $R = \log_2 L$ bit per campione.

Mean Square Error (MSE) come indice della distorsione

Si definisce l'errore di quantizzazione $e(n)$ come la differenza tra il campione del segnale campionato $s_c(n)$ e il campione corrispondente del segnale quantizzato $s_q(n)$:

$$e(n) = s_c(n) - s_q(n)$$

Il MSE (Mean Square Error) è un indice della distorsione dovuta alla quantizzazione, corrisponde all'errore quadratico medio ed equivale alla norma euclidea al quadrato del vettore errore o alla distanza euclidea al quadrato tra il vettore dei campioni del segnale campionato e il vettore dei campioni del segnale quantizzato (al netto della normalizzazione per il fattore $1/N$).

$$\text{MSE}(s_c(n), s_q(n)) = \frac{1}{N} \sum_{n=0}^{N-1} [e(n)]^2 = \frac{1}{N} \sum_{n=0}^{N-1} [s_c(n) - s_q(n)]^2 = \frac{1}{N} \|\vec{e}\|^2 = \frac{1}{N} \|\vec{s}_c - \vec{s}_q\|^2$$

Peak Signal-to-Noise Ratio (PSNR) come indice della qualità

Il PSNR (Peak Signal-to-Noise Ratio) è un indice della qualità del segnale quantizzato, viene calcolato come rapporto in dB tra la potenza di picco del segnale e il MSE. Il PSNR e il MSE sono strettamente legati tra di loro e totalmente definiti l'uno dall'altro. Il PSNR non corrisponde esattamente alla qualità percepita da un osservatore umano, però viene spesso utilizzato data la sua semplicità di calcolo.

$$\text{PSNR} = 10 \log_{10} \frac{(2^b - 1)^2}{\text{MSE}}$$

Di seguito è riportato un esempio di calcolo del PSNR per un'immagine in scala di grigi quantizzata a 8 bit con 255 livelli di grigio, con alcune indicazioni sulla qualità percepita in base al valore del PSNR.

$$\text{PSNR} = 10 \log_{10} \frac{255^2}{\text{MSE}} = 48\text{dB} - 10 \log_{10} \text{MSE}$$

- PSNR > 45 dB: ottima qualità, distorsioni non percepibili
- 40 dB < PSNR < 45 dB: qualità molto buona, distorsioni difficilmente percepibili
- 35 dB < PSNR < 40 dB: qualità buona, distorsioni percepibili
- 30 dB < PSNR < 35 dB: qualità accettabile, distorsioni evidenti
- PSNR < 30 dB: qualità scadente

Assenza di un blocco inverso alla quantizzazione

Siccome la quantizzazione è un'operazione irreversibile, non esiste alcun processo inverso in grado di ricostruire il segnale campionato originale a partire dal segnale quantizzato. Per questo motivo, durante la fase di ricostruzione del segnale analogico, non ci sarà alcun blocco inverso alla quantizzazione.

2.4 Inverse bit mapper (IBM) e parallel-to-serial converter (P/S)

Il quantizzatore produce in output simboli discreti che coincidono con i vari livelli di quantizzazione. Tali simboli vengono poi convertiti in vettori (o sequenze) di bit tramite il modulo Inverse Bit Mapper (IBM).

L'IBM produce in output vettori di bit in parallelo, per cui è necessario un altro modulo detto Parallel to Serial Converter (P/S) per convertirli in sequenze seriali che potranno poi essere codificate e trasmesse.

Il bitrate del blocco di conversione ADC è dato dal prodotto tra il tasso di codifica R del quantizzatore e la frequenza di campionamento F_c del segnale campionato. $R_{\text{ADC}} = F_c \cdot R_Q = F_c \cdot \log_2 L$

2.5 Serial-to-parallel converter (S/P) e bit mapper (BM)

Una volta che il ricevitore riceve i simboli trasmessi, questi vengono decodificati in sequenze seriali di bit. Tali sequenze vengono poi convertite in vettori di bit in parallelo tramite il modulo Serial to Parallel Converter (S/P) per poi essere trasformate in simboli discreti tramite il modulo Bit Mapper (BM). In questo modo si riottiene una ricostruzione del segnale quantizzato originale che poi verrà interpolato per ottenere il segnale analogico ricostruito.

2.6 Interpolazione

Formula di interpolazione ideale

Dal teorema di Shannon-Nyquist, per ricostruire un segnale analogico a partire dai campioni del segnale campionato secondo opportune condizioni, si utilizza la formula di interpolazione ideale:

$$s(t) = \sum_{n \in \mathbb{Z}} s(nT_c) \cdot \text{sinc}\left(\frac{t - nT_c}{T_c}\right)$$

Si nota però che tale formula non è implementabile in pratica, in quanto richiede di sommare un numero infinito di segnali $\text{sinc}(t)$ a supporto infinito. Nella pratica si sostituisce la $\text{sinc}(t)$ con altri segnali a supporto finito detti kernel di interpolazione.

Kernel di interpolazione e vincoli

Un kernel di interpolazione è un segnale a supporto finito (implementabile nella pratica) che viene utilizzato al posto del kernel ideale $\text{sinc}(t)$. La funzione di interpolazione diventa:

$$s(t) = \sum_{n \in \mathbb{Z}} s(nT_c) \cdot h\left(\frac{t - nT_c}{T_c}\right) \quad \text{con } h\left(\frac{t - nT_c}{T_c}\right) \begin{array}{l} \text{kernel di interpolazione dilatato} \\ \text{nell'intervallo di camp. } [-T_c, T_c] \text{ e} \\ \text{traslato nell'istante di camp. } nT_c \end{array}$$

I kernel di interpolazione devono rispettare alcuni vincoli per la conservazione dei valori negli istanti di campionamento per una corretta ricostruzione del segnale campionato, come illustrato di seguito.

$$h(n) = \begin{cases} 1 & \text{se } n = 0 \\ 0 & \text{se } n \in \mathbb{Z} \setminus \{0\} \end{cases} \implies \tilde{s}(kT_c) = \sum_{n \in \mathbb{Z}} s(nT_c) \cdot h\left(\frac{kT_c - nT_c}{T_c}\right) = \sum_{n \in \mathbb{Z}} s(nT_c) \cdot h(k - n) = s(kT_c)$$

Esempi di kernel di interpolazione

- **sample and hold**, con kernel di interpolazione a gradino rettangolare:

$$h(t) = \text{rect}(t - 1/2) = \begin{cases} 1 & \text{se } 0 \leq t < 1 \\ 0 & \text{altrimenti} \end{cases}$$

il segnale ricostruito è un segnale a gradino che mantiene il valore del campione per tutto l'intervallo di campionamento, è il kernel di interpolazione più semplice da implementare, ma introduce distorsioni evidenti e alte frequenze spurie dovute alla discontinuità delle funzioni di interpolazione, tali artefatti risultano accettabili solo se la f_c è elevata

- **interpolazione lineare** o di primo ordine, con kernel di interpolazione a triangolo:

$$h(t) = \text{triang}(t) = \begin{cases} 1 - |t| & \text{se } |t| < 1 \\ 0 & \text{altrimenti} \end{cases}$$

il segnale ricostruito corrisponde ad una linea spezzata che unisce i valori dei vari campioni, è più complesso da implementare, ma introduce meno distorsioni e frequenze spurie rispetto al sample and hold

- **interpolazione cubica** o di terzo ordine, con kernel di interpolazione polinomiale di terzo grado: si effettua un'interpolazione con polinomi di terzo grado tra i campioni, è più complesso da implementare e spesso si usa negli algoritmi di interpolazione digitale delle immagini (es. cambio di risoluzione di un'immagine) in quanto esistono algoritmi efficienti basati sul filtraggio numerico
- **interpolazione con spline cardinali** con kernel di interpolazione polinomiale di grado n : è possibile usare come kernel di interpolazione un generico polinomio di grado n che implica la continuità delle derivate fino all'ordine $n-1$ del segnale ricostruito, riduce sempre di più le distorsioni e le frequenze spurie, ma risulta sempre più complesso da implementare, per $n \rightarrow \infty$ coincide con l'interpolazione ideale a $\text{sinc}(t)$
- **interpolazione con sinc troncato** con kernel di interpolazione a sinc troncato:

$$h(t) = \text{sinc}(t) \cdot \text{rect}\left(\frac{t}{T}\right) = \begin{cases} \frac{\sin(\pi t)}{\pi t} & \text{se } |t| < T \\ 0 & \text{altrimenti} \end{cases}$$

è il kernel utilizzato all'interno delle schede audio di fascia alta, in quanto rappresenta un'ottima approssimazione fisicamente realizzabile del kernel ideale $\text{sinc}(t)$; il parametro T viene scelto in fase di progettazione, maggiore è T , maggiore è la fedeltà della ricostruzione ed aumenta la complessità computazionale in quanto cresce il numero di campioni coinvolti per la ricostruzione del segnale in ogni istante di tempo t

3 Rappresentazione dei segnali multimediali

3.1 Rappresentazione di segnali audio

Percezione del suono

Il sistema uditivo umano percepisce i suoni effettuando un'analisi in frequenza del segnale audio che giunge all'orecchio. Può essere modellato come un banco di filtri che scompongono il suono in più bande di frequenza, ciascuna delle quali è associata ad una fibra nervosa che trasmette l'informazione al cervello. L'ascolto di un tono puro attiverà un solo filtro che ecciterà la fibra nervosa corrispondente.

Soglia di percezione acustica

La soglia di percezione acustica corrisponde al livello minimo di potenza sonora che può essere percepito dall'orecchio umano. Varia in funzione della frequenza del suono e cambia da persona a persona, in base all'età. In generale la soglia di percezione acustica è più bassa (ovvero l'orecchio umano è più sensibile) per le frequenze comprese tra 500 Hz e 5 kHz che corrispondono alla fascia del parlato umano.

La soglia di pressione acustica non è sempre costante, ma in funzione dei suoni percepiti attraverso due fenomeni di mascheramento:

- **mascheramento in frequenza:** la presenza di un suono forte in una certa banda di frequenza fa aumentare la soglia di percezione acustica dei suoni alle frequenze adiacenti, per cui un suono debole nella stessa banda di frequenza può non essere percepito
- **mascheramento temporale:** la modifica della curva di soglia di percezione acustica dovuta alla presenza di un suono forte in una certa banda di frequenza si propaga nel tempo per 2-5ms prima del suono forte e per 100-200ms dopo il suono forte, per cui un suono debole in una banda di frequenza può non essere percepito se è stato appena emesso un suono forte in una banda di frequenza vicina

Codifica percettiva

La codifica percettiva sfrutta le caratteristiche del sistema uditivo umano e i fenomeni di mascheramento per ridurre la quantità di dati necessaria a rappresentare un segnale audio senza introdurre distorsioni percepibili dall'orecchio umano. Si può realizzare attraverso un'analisi tempo-frequenza che applica riquantizzazioni alle frequenze mascherate.

Digitalizzazione tramite Pulse Code Modulation (PCM)

La codifica PCM consiste nella digitalizzazione di un segnale audio analogico attraverso le sole fasi essenziali di campionamento, quantizzazione e inverse bit mapping. Non vengono applicate compressioni, per questo viene anche detta codifica lossless. Gli unici parametri che influenzano la qualità del segnale digitale sono, quindi, la frequenza di campionamento f_c e il numero di livelli di quantizzazione L .

Digitalizzazione di un segnale vocale

Per digitalizzare un segnale vocale con banda [200 Hz, 3.4 kHz], si utilizza una frequenza di campionamento $f_c = 8$ kHz e un quantizzatore da 256 livelli con 8 bpS (128 livelli con 7 bpS in Asia e America). Eventualmente si suddividono i dati in pacchetti di 20 ms da 1.28 Mbit. Il bitrate risulta essere:

$$R_{\text{voce}} = f_c \cdot \log_2 L = 8 \text{ kHz} \cdot 8 \text{ bpS} = 64 \text{ kbit/s} \quad \text{packet size} = 20 \text{ ms} \cdot R_{\text{voce}} = 1280 \text{ bit} = 160 \text{ byte}$$

Digitalizzazione di un segnale audio generico

Per digitalizzare un segnale audio stereo generico (es. musica) con banda [15 Hz, 20 kHz] si utilizza una frequenza di campionamento di $f_c = 44.1$ kHz con 16 bpS, duplicato per i due canali. Il bitrate è di:

$$R_{\text{musica}} = 2 \cdot f_c \cdot \log_2 L = 2 \cdot 44.1 \text{ kHz} \cdot 16 \text{ bpS} = 1.411 \text{ Mbit/s}$$

Memorizzazione dei segnali codificati con PCM nei CD

Nei CD audio, l'informazione audio memorizzata è codificata tramite codifica PCM, e comprende solo l'87% dei dati memorizzati. Il restante 13% è occupato dai codici di correzione d'errore (CRC). Il bitrate effettivo risulta essere:

$$R_{CD} = R_{musica} / 0.87 = 1.622 \text{ Mbit/s}$$

Se si volesse calcolare la massima durata effettiva di un CD audio da 730 MB si esegue il seguente calcolo:

$$\text{durata} = \frac{\# \text{ bit disponibili}}{\text{bit/s effettivi}} = \frac{730 \text{ MB} \cdot 8 \cdot 10^6 \text{ bit/MB}}{1.622 \cdot 10^6 \text{ bit/s}} \approx 3600 \text{ s} = 1 \text{ h}$$

3.2 Rappresentazione di immagini

Rappresentazione di immagini B/N - campionamento spaziale e HDR

La fase di acquisizione della luce di una scena reale tramite un sensore digitale (es. sensore CMOS) consiste in un campionamento spaziale della luce che colpisce il sensore in ogni pixel di cui esso è costituito. Ogni pixel misura l'intensità della luce che lo colpisce e la converte in un valore numerico, solitamente quantizzato su 8 bit da 0 a 255. Il risultato è una matrice di valori numerici che rappresentano l'immagine acquisita in scala di grigi (B/N).

Esistono sensori digitali con gamma dinamica estesa e rappresentano le immagini in formato HDR in grado di supportare fino a 10-12 bit per pixel. Tali tecnologie non vanno, però confuse con gli algoritmi che simulano l'HDR combinando più immagini della stessa scena con esposizioni differenti.

Percezione del colore

L'occhio umano percepisce la luce e i colori attraverso due tipi di cellule presenti sulla retina:

- i bastoncelli, sono in numero maggiore e sono sensibili alla intensità luminosa
- i coni, sono in numero minore e sono sensibili a tre bande di frequenza della luce corrispondenti ai colori primari rosso (575 nm), verde (535 nm) e blu (445 nm)

Si osserva, quindi, che è sufficiente memorizzare il contributo di ciascun colore primario per rappresentare un qualsiasi colore percepito dall'occhio umano. Inoltre, essendo i coni in minoranza rispetto ai bastoncelli, l'occhio umano è più sensibile alle variazioni di luminosità che alle variazioni di colore.

Rappresentazione di immagini a colori - RGB

La rappresentazione di immagini a colori tramite il formato RGB consiste nel memorizzare il contributo di ciascun colore primario (R, G, B) per ogni pixel dell'immagine. Ogni canale di colore è quantizzato su 8 bit con valori da 0 a 255, per cui complessivamente ogni pixel è rappresentato da 24 bit e può codificare fino a $2^{24} = 16\,777\,216$ colori diversi.

Anche per le immagini a colori esistono formati con gamma dinamica estesa (HDR) che supportano fino a 12 bit per canale di colore, con 36 bit per pixel e $2^{36} = 68\,719\,476\,736$ colori rappresentabili.

Rotazione dallo spazio RGB allo YCbCr

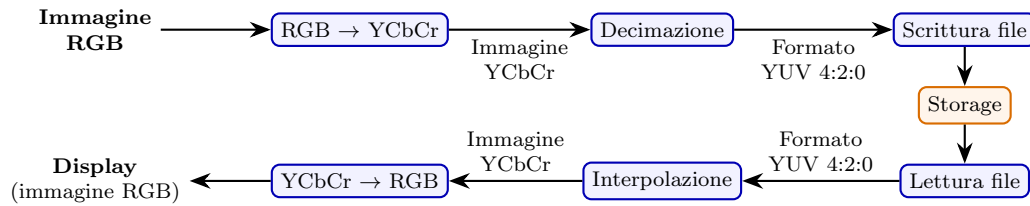
Applicando un'opportuna rotazione degli assi, è possibile avere la diagonale principale $(0, 0, 0) \rightarrow (1, 1, 1)$ dello spazio RGB come coordinata Y, con altre due coordinate ortogonali ad essa dette Cb e Cr. La coordinata Y è detta luminanza del colore, infatti sulla diagonale principale si trovano tutte le tonalità di grigio, mentre le coordinate Cb e Cr sono il complemento al blu e il complemento al rosso e rappresentano la componente cromatica detta anche cromaticanza del colore sul piano ortogonale alla luminanza.

Sparsificazione del segnale YCbCr

Un segnale viene detto sparso quando presenta alcuni campioni "importanti" e altri campioni "meno importanti" che è possibile comprimere con apposite tecniche di sottocampionamento senza degradare eccessivamente il segnale rappresentato.

Il segnale YCbCr viene detto sparso in quanto l'occhio umano è più sensibile alla luminanza Y, rispetto al contributo cromatico Cb e Cr. È possibile, quindi, applicare un processo di decimazione dei campioni Cb e Cr, riducendo il costo di ogni pixel senza introdurre distorsioni percepibili dall'occhio umano.

Pipeline di conversione da RGB a YCbCr e viceversa



Il processo di conversione da RGB a YCbCr e viceversa prevede i seguenti passaggi:

1. conversione da RGB a YCbCr tramite una trasformazione lineare che ruota lo spazio dei colori, è implementabile come un semplice prodotto tra la matrice di trasformazione e il vettore RGB di ogni pixel dell'immagine
2. decimazione dei campioni Cb e Cr, tramite il processo di chroma subsampling descritto in seguito
3. memorizzazione in memoria o invio al destinatario del file compresso
4. interpolazione per ricostruire i campioni Cb e Cr decimati, tramite un filtro di interpolazione
5. conversione da YCbCr a RGB tramite la trasformazione lineare inversa

Decimazione per sottocampionamento della cromaticità (chroma subsampling)

Il processo di decimazione dei campioni Cb e Cr viene effettuato tramite il formato YUV J:a:b. In pratica si suddivide l'immagine in blocchi di 4×2 pixel e i tre valori J, a e b indicano quanti campioni di ciascun canale vengono memorizzati per ogni blocco. Di seguito alcuni formati di decimazione più diffusi, l'attuale standard universale è il 4:2:0, utilizzato nella quasi totalità dei formati foto e video attualmente esistenti.

- **4:4:4**: non viene effettuata alcuna decimazione
- **4:2:2**: per ogni riga si memorizzano solo 4 campioni di Y, 2 di Cb e 2 di Cr, riducendo la banda del colore del 50% e il costo complessivo della rappresentazione del 33.3%
- **4:2:0**: per ogni mezzo blocco di 2×2 pixel si memorizzano solo 4 campioni di Y, 1 di Cb e 1 di Cr, riducendo la banda del colore del 75% e il costo complessivo della rappresentazione del 50%
- **4:1:1**: per ogni riga si memorizzano solo 4 campioni di Y, 1 di Cb e 1 di Cr, riducendo la banda del colore del 75% e il costo complessivo della rappresentazione del 50%

In generale un'immagine HD di risoluzione 1920×1080 rappresentata in formato RGB occupa approssimativamente 6.22 MBytes. Applicando la trasformazione da RGB a YCbCr e il sottocampionamento 4:2:0, il costo scende a 3.11 MBytes, senza introdurre distorsioni percepibili dall'occhio umano.

$$\text{size}(X_{\text{RGB}}) = 1920 \times 1080 \text{ pixel} \cdot 3 \text{ canali} \cdot 8 \text{ bit} \approx 49.77 \text{ Mbit} = 6.22 \text{ MByte}$$

$$\text{size}(X_{\text{YCbCr}}) = 1920 \times 1080 \text{ pixel} \cdot 3 \text{ canali} \cdot 8 \text{ bit} / 2 \approx 24.88 \text{ Mbit} = 3.11 \text{ MByte}$$

3.3 Rappresentazione di video

Percezione del movimento

Il fenomeno della persistenza retinale consiste nel fatto che un'immagine osservata permane per un breve periodo di tempo sulla retina dell'occhio umano. Questo permette di campionare nel tempo un segnale video e di visualizzare le immagini campionate in rapida successione, dando l'illusione di un movimento continuo. Le immagini campionate sono dette fotogrammi o frame e solitamente vengono visualizzate a 24, 30, 50 o 60 frame al secondo (fps).

Formati video non compressi

Per un video non compresso, il bitrate è estremamente elevato e si richiederanno altre tecniche di compressione per poterlo trasmettere efficacemente in rete. Ad esempio un video HD di risoluzione 1920×1080 con 50 fps con frame codificati in formato YCbCr 4:2:0 ha un bitrate di circa 155 MBytes/s.

$$R_{\text{YCbCr}} = \text{frame-size}_{\text{YCbCr}} \cdot 50 \text{ fps} = 3.11 \text{ MBytes} \cdot 50 \text{ fps} \approx 1.244 \text{ Gbit/s} = 155.5 \text{ MBytes/s}$$

4 Codifica vocale

4.1 Caratteristiche della voce e classi di codificatori

Suoni vocali e non vocali

Il segnale vocale può essere considerato un segnale localmente stazionario su periodi di 20 ms. In tali periodi, il carattere del segnale rimane costante per la stazionarietà durante la pronuncia di un fonema. In generale i periodi di stazionarietà si classificano in due categorie principali:

- suoni vocali: segnali pseudoperiodici, associati alle vocali e alle consonanti s e f, caratterizzati da alcune frequenze predominanti dovute alla periodicità data dalla vibrazione delle corde vocali
- suoni non vocali: segnali aperiodici, associati alle consonanti p, t, k, s, f, h, caratterizzati da uno spettro piuttosto uniforme, simile al rumore bianco
- suoni misti: transizioni tra i diversi fonemi

Classi di codificatori

- **codificatori a forme d'onda:** codificano la forma d'onda del segnale, senza ottimizzazioni per le caratteristiche del segnale vocale; hanno bassa complessità, alta qualità del parlato, ma bitrate elevato; un esempio è la codifica PCM
- **codificatori vocali o vocoder:** modellano il modo in cui il segnale vocale viene generato, stimando i parametri del modello e trasmettendo solo questi parametri; hanno bassa qualità del parlato, ma bitrate molto basso; un esempio è la codifica LPC-10
- **codificatori ibridi:** combinano le due tecniche precedenti, utilizzano tecniche di vocoding unite a codifiche a forme d'onda per il residuo di predizione; hanno qualità del parlato migliore dei vocoder e bitrate più elevato; esempi sono ACELP, AMR-WB e Opus

4.2 Codificatore vocoder basato su LPC-10

Codifica a predizione lineare (LPC-10)

La codifica a predizione lineare LPC-10 consiste nel produrre una predizione $x_p(n)$ del campione corrente $x(n)$ attraverso una combinazione lineare dei 10 campioni precedenti $x(n-1)$, $x(n-2)$, ..., $x(n-10)$ pesati da opportuni coefficienti $a_1, a_2, \dots, a_{10}$.

$$x_p(n) = \sum_{k=1}^{10} a_k x(n-k)$$

I coefficienti a_k vengono calcolati minimizzando l'errore di predizione quadratico medio tra il campione corrente $x(n)$ e la sua predizione $x_p(n)$. L'errore di predizione $y(n)$ è definito come segue:

$$y(n) = x(n) - x_p(n) = x(n) - a_1 x(n-1) - a_2 x(n-2) - \dots - a_{10} x(n-10)$$

Si osserva che conoscendo i coefficienti a_k e l'errore di predizione $y(n)$, è possibile ricostruire esattamente il campione corrente $x(n)$. Risulta sufficiente che il codificatore trasmetta tali informazioni al decodificatore (opportunamente quantizzate) per permettere la ricostruzione del segnale vocale.

Trasmissione del residuo

Se i coefficienti a_k sono stati calcolati correttamente, il residuo $y(n)$ risulta essere:

- un fruscio o rumore bianco per i suoni non vocali, di cui viene trasmesso solo la potenza, siccome il decodificatore è in grado di generare da sé un rumore bianco casuale
- un segnale pseudoperiodico con frequenza predominante detta pitch per i suoni vocali, che viene trasmesso al decodificatore come treno di impulsi periodici

Per individuare pitch e potenza del residuo si utilizzano tecniche come la stima dell'autocorrelazione.

Tasso di codifica

Supponendo di suddividere il segnale vocale in finestre di 20 ms, per ogni finestra si trasmettono:

- 36 bit per i 10 coefficienti del filtro di predizione lineare
- 1 bit per distinguere se l'errore di predizione è un treno di impulsi o rumore bianco
- 6 bit per la potenza del residuo, sia per il rumore bianco che per il treno di impulsi
- 7 bit per il pitch del residuo, solo se l'errore di predizione è un treno di impulsi

Complessivamente si ottiene un tasso di codifica medio di 2.4 kbps, che è molto più basso rispetto ai 64 kbps della codifica PCM, a discapito di una qualità del parlato molto più bassa.

$$R_{\text{voiced}} = \frac{36 + 1 + 6 + 7 \text{ bit}}{20 \text{ ms}} = 2500 \text{ bps} \quad R_{\text{unvoiced}} = \frac{36 + 1 + 6 \text{ bit}}{20 \text{ ms}} = 2150 \text{ bps} \quad R_{\text{avg}} = 2400 \text{ bps}$$

Criticità del codificatore LPC-10

Il codificatore LPC-10 ha alcune criticità che lo rendono poco utilizzabile in applicazioni pratiche:

- **decisione binaria:** la classificazione dei campioni in “vocali” e “non vocali” risulta troppo stringente in quanto non tiene conto degli stati intermedi di transizione tra i fonemi; sarebbe opportuno avere a disposizione più classi di campioni per catturare meglio la variabilità del segnale vocale
- **errori di classificazione:** se il codificatore commette errori nel classificare un campione, si verificheranno distorsioni e perdita di informazioni nel segnale ricostruito creando un effetto “sussurrato” o “soffiato” nel parlato; per risolvere ciò si possono utilizzare modelli con catene di feedback
- **semplificazione del residuo:** trattando il residuo come generico rumore bianco, si perdono informazioni importanti sulla struttura spettrale e sulla coerenza di fase del segnale originale, producendo un timbro artificiale; risulta utile avere più parametri per modellare meglio il residuo
- **latenza:** i codificatori a predizione lineare possiedono una certa latenza dovuta in parte al costo computazionale, in parte ad eventuali buffer necessari nelle implementazioni pratiche
- **resilienza:** i codificatori a predizione lineare sono molto sensibili agli errori casuali e agli errori a burst, per cui è necessario impiegare opportune tecniche di FEC (Forward Error Correction)
- **soppressione del silenzio:** per risparmiare banda, si può aggiungere un modulo di rilevamento dell'attività vocale (VAD, Voice Activity Detection) che interrompe la trasmissione del segnale durante i periodi di silenzio, ma allo stesso tempo può portare a improvvise interruzioni del parlato

4.3 Codificatori più avanzati - ACELP, AMR-WB e Opus

Codificatore ACELP - G.729

Il codificatore ACELP (Algebraic Code-Excited Linear Prediction) è un codificatore basato su predizione lineare che dispone di un set molto esteso di residui (generati da codici algebrici) per eccitare il filtro di predizione. Ciò permette di modellare meglio gli stati intermedi di transizione tra fonemi e dispone di feedback loop per migliorare le prestazioni. Possiede anche moduli per la gestione dei mascheramenti.

Codificatore AMR-WB - G.722.2

Il codificatore Adaptive Multi-Rate Wideband (AMR-WB) è un codificatore ibrido utilizzato come standard per l'HD-Voice nelle reti mobili LTE/5G. La particolarità di questo codificatore è l'estensione della banda passante fino a 7 kHz (contro i 4 kHz della banda telefonica tradizionale) e la possibilità di variare il bitrate da 6.6 kbps a 23.85 kbps in base alla qualità del canale radio. Questo permette maggiore qualità del parlato grazie all'aumento del contenuto armonico delle alte frequenze e una maggiore robustezza della trasmissione in caso di canali radio con banda limitata.

4.4 Codificatore Opus - RFC 6716

Caratteristiche generali

Il codificatore Opus è un codificatore ibrido con bitrate variabile da 6 kbps a 510 kbps e banda estesa da 8 kHz a 48 kHz, ampiamente utilizzato per servizi di videochiamata WebRTC (meet, zoom, teams), messaggistica (whatsapp, telegram), streaming audio (youtube, spotify) e gaming (discord, twitch).

Architettura ibrida SILT + CELT

Opus si basa su due motori di elaborazione paralleli per codificare il segnale vocale e il segnale musicale:

- **motore SILT**: utilizza la predizione lineare (LPC) basata sul modello del tratto vocale umano, come visto in precedenza; si occupa della parte inferiore dello spettro armonico (≤ 8 kHz) tipica del parlato
- **motore CELT**: utilizza una variante ottimizzata della trasformata discreta di Fourier (MDCT), simile a quella utilizzata nei codificatori MP3 e AAC; si occupa di codificare la parte superiore dello spettro armonico (> 8 kHz) tipica della musica e in generale dei suoni non vocali

Se il segnale è prevalentemente vocale, il codificatore utilizza solo il motore SILT, mentre se il segnale contiene anche componenti musicali, il codificatore utilizza entrambi i motori in parallelo. In questo modo, si riesce a garantire una qualità del parlato molto elevata con bassi bitrate e contemporaneamente una qualità musicale paragonabile a quella dei codificatori MP3 e AAC.

Diffusione e vantaggi su altri codificatori

La sua grande diffusione è dovuta al fatto che ha una bassa latenza algoritmica e strutturale di soli 26.5 ms, dispone di grande flessibilità nella gestione del bitrate e ha una banda sufficientemente estesa sia per segnali vocali che musicali. Inoltre è open source e royalty-free, per cui può essere utilizzato liberamente senza costi di licenza da chiunque necessiti di un codificatore universale di qualità.

5 Codifica lossless

5.1 Introduzione alle codifiche lossless

La codifica lossless consiste nel rappresentare una sequenza di simboli emessi da una sorgente tramite una stringa di bit in modo perfettamente invertibile, cioè senza perdita di informazione. L'obiettivo delle codifiche lossless è quello di rappresentare i simboli nel minor numero di bit possibile (preservando l'informazione) in modo da ridurre la quantità di dati da trasmettere o memorizzare.

Definizioni di base

$\mathcal{X} = \{x_1, x_2, \dots, x_M\}$	alfabeto con M simboli emessi da una sorgente X , ciascuno con la sua probabilità $p_i = P(X = x_i)$
$C : x_i \in \mathcal{X} \rightarrow c_i \in \{0, 1\}^*$	codice C definito come funzione di codifica che associa una stringa di bit (o codeword) c_i a ciascun simbolo x_i ogni codeword ha la sua lunghezza $l_i = c_i $
$\mathcal{L} = \sum_{i=1}^M p_i l_i$	lunghezza media del codice calcolata come media delle lunghezze di ciascuna codeword pesate per le relative probabilità dei simboli

Codici univocamente decodificabili

Per garantire assenza di perdita di informazione, il codice deve essere univocamente decodificabile, cioè ogni sequenza di codewords senza separatori deve essere decodificabile in un'unica sequenza di simboli. Di seguito alcuni esempi di codici univocamente decodificabili:

- **fixed length coding - FLC**: ogni simbolo è associato ad una codeword di lunghezza fissa, la decodifica è estremamente facile, ma non è ottimale in quanto non tiene conto della distribuzione di probabilità dei simboli
- **prefix coding**: nessuna codeword è prefisso di un'altra codeword, quindi ogni volta che si legge una codeword, la si può decodificare immediatamente essendo certi che la lettura dei successivi bit non porterà a una codeword più lunga, permette di ottenere codici ottimi se si tiene conto della distribuzione di probabilità dei simboli
- **unary coding**: codice in cui le codewords sono sequenze di 1 (o 0) delimitate all'inizio o alla fine da un bit 0 (o 1), la decodifica consiste nel contare il numero di bit uguali della sequenza prima di incontrare il bit delimitatore, che indica la fine della codeword o l'inizio della successiva

Entropia di una variabile aleatoria

Data una variabile aleatoria X che assume valori discreti x_i di un alfabeto $\mathcal{X}$, ciascuno con probabilità $p_i = P(X = x_i)$, si definiscono:

$$I(x_i) = \log_2 \frac{1}{p_i} = -\log_2 p_i \quad \text{informazione associata all'evento } X = x_i$$

$$H(X) = E[I(X)] = \sum_{i=1}^M p_i I(x_i) = \sum_{i=1}^M p_i \log_2 \frac{1}{p_i} \quad \text{entropia della variabile aleatoria } X$$

L'entropia rappresenta la misura della quantità di informazione media contenuta in un simbolo emesso dalla sorgente e può essere interpretata come il grado di incertezza associato alla variabile aleatoria X . L'entropia è massima quando gli M simboli sono equiprobabili, con $p_i = 1/M$, e vale $H(X) = \log_2 M$. Viceversa quando si hanno pochi simboli molto probabili e molti simboli poco probabili (distribuzioni sparse), l'entropia è molto bassa.

Teorema di Shannon sulla codifica di sorgente

La lunghezza media $\mathcal{L}^*$ di un codice decodificabile ottimo per una sorgente X è vincolata dalla relazione:

$$H(X) \leq \mathcal{L}^* < H(X) + 1 \quad \text{con } \mathcal{L}^* = H(X) \iff p_i = 2^{-l_i} \text{ (per distribuzioni diadiche)}$$

Codifica a blocchi

Per sfruttare la correlazione tra simboli consecutivi emessi dalla sorgente, è possibile raggruppare K simboli consecutivi in un blocco e codificare ogni blocco come un simbolo unico.

Analizzando la formula del teorema di Shannon, si osserva che la lunghezza media per simbolo $\mathcal{L}_S^*$ del codice tende a convergere verso il limite teorico dettato dall'entropia all'aumentare di K , siccome il bit di overhead +1 viene distribuito sui K simboli di ogni blocco.

$$H(X^K) \leq L^* < H(X^K) + 1 \implies \frac{H(X^K)}{K} \leq \mathcal{L}_S^* < \frac{H(X^K)}{K} + \frac{1}{K} \quad \text{con } \mathcal{L}_S^* = \frac{\mathcal{L}^*}{K}$$
$$\lim_{K \rightarrow \infty} \frac{H(X^K)}{K} = \lim_{K \rightarrow \infty} \frac{H(X^K)}{K} + \frac{1}{K} = \lim_{K \rightarrow \infty} \mathcal{L}_S^*$$

Tasso entropico come limite assoluto delle codifiche lossless

Si osserva inoltre che l'entropia per blocco $H(X^K)$ decresce all'aumentare della lunghezza del blocco K o al più rimane costante se i simboli presi singolarmente sono indipendenti. Di conseguenza anche l'entropia per simbolo $H(X^K)/K$ e la relativa lunghezza media per simbolo $\mathcal{L}_S^*$ decrescono fino a raggiungere il limite teorico assoluto per le codifiche lossless detto tasso entropico $\mathcal{H}(X)$ della sorgente X .

$$\mathcal{H}(X) := \lim_{K \rightarrow \infty} \frac{H(X^K)}{K} = \lim_{K \rightarrow \infty} \mathcal{L}_S^*$$

Idealmente si vorrebbe utilizzare blocchi di lunghezza sempre maggiore, solo che aumentando K aumenta anche la complessità computazionale della codifica e lo spazio necessario per memorizzare le codewords associate a ciascun blocco. Nella pratica serve scegliere il giusto compromesso tra lunghezza del blocco K e complessità computazionale.

5.2 Codifica di Huffman

Processo di codifica

La codifica di Huffman è un algoritmo di compressione lossless che permette di ottenere uno dei codici ottimi per codificare i simboli emessi da una sorgente discreta. L'algoritmo costruisce un albero binario in cui i simboli compongono le foglie e le codewords sono ottenute unendo le etichette 0 e 1 degli archi che compongono i cammini dalla radice alle foglie.

Criticità della codifica per simboli

La codifica di Huffman presenta due principali criticità:

- la lunghezza minima delle codewords è 1, quindi la lunghezza media del codice non può essere inferiore a 1, di conseguenza se l'entropia della sorgente è prossima allo zero (segnali molto sparsi e prevedibili), la codifica di Huffman non è ottimale
- la codifica di Huffman si basa solo sulla distribuzione di probabilità dei simboli, ma non tiene conto di possibili correlazioni tra simboli consecutivi emessi dalla sorgente, quindi non è ottimale per sorgenti in cui si hanno correlazioni tra simboli consecutivi (ad esempio pixel adiacenti di un'immagine)

Per risolvere tali criticità si possono utilizzare le codifiche a blocchi, al costo di un aumento della complessità esponenziale in funzione di K . Per questo si utilizzano codifiche alternative per avvicinarsi al limite teorico del tasso entropico della sorgente, come la codifica aritmetica o la codifica basata su dizionari.

5.3 Codifica aritmetica

Processo di codifica per simboli

La codifica aritmetica consiste nel suddividere l'intervallo $[0, 1]$ in sottointervalli proporzionali alle probabilità dei vari simboli emessi dalla sorgente. La codeword associata ad ogni simbolo è ottenuta convertendo in binario il numero reale che rappresenta il punto medio del sottointervallo corrispondente al simbolo.

Estensione per codifica a blocchi

Tale sistema risulta molto vantaggioso per codifiche a blocchi, in quanto per estendere i blocchi di simboli consecutivi basta suddividere ricorsivamente il sottointervallo corrispondente al primo simbolo della sequenza con la distribuzione di probabilità dei possibili simboli successivi. Tale distribuzione di probabilità può essere uguale a quella dei simboli presi singolarmente (utile nel caso di simboli indipendenti) oppure è possibile usare le probabilità condizionate per avvicinarsi al limite teorico del tasso entropico della sorgente.

Codifica adattiva

A differenza di Huffman che richiede di conoscere a priori la distribuzione di probabilità dei simboli, la codifica aritmetica permette di aggiornare dinamicamente il codice modificando le distribuzioni di probabilità e riadattando i sottointervalli in base alla frequenza con cui i simboli vengono emessi.

Analisi delle prestazioni

Il codice aritmetico per simboli singoli non è ottimale in quanto il limite superiore è più largo rispetto al codice di Huffman, come indicato di seguito:

$$\mathcal{L}_A < H(X) + 2$$

Applicando la codifica aritmetica a blocchi di K simboli consecutivi, però, risulta più vantaggioso rispetto al codice di Huffman in quanto la complessità aumenta linearmente con K , e la lunghezza media per simbolo converge sempre al tasso entropico della sorgente, come indicato di seguito:

$$\mathcal{L}_{A,S}^K < \frac{H(X^K)}{K} + \frac{2}{K} \quad \text{con} \quad \mathcal{L}_{A,S}^K = \frac{\mathcal{L}_A}{K} \quad \lim_{K \rightarrow \infty} \mathcal{L}_{A,S}^K = \lim_{K \rightarrow \infty} \frac{H(X^K)}{K} = H(X)$$

Lo svantaggio principale è che la codifica aritmetica ha una complessità computazionale maggiore in quanto richiede di svolgere operazioni tra numeri floating point con precisione sempre maggiore all'aumentare di K e necessita di tecnologie proprietarie.

5.4 Codifiche basate su dizionari - Lempel-Ziv, DEFLATE e ASN

Funzionamento della codifica basata su dizionari (LZ77, LZ78, LZW)

La codifica basata su dizionari sviluppata dagli algoritmi Lempel-Ziv (LZ77, LZ78) e Lempel-Ziv-Welch (LZW) si basa sul riconoscimento di pattern e sequenze di simboli ripetute. In pratica le sequenze di simboli ricorrenti vengono sostituite con un puntatore ad un dizionario o ad un punto precedente del messaggio stesso in cui è presente tale stringa.

Algoritmo DEFLATE

L'algoritmo DEFLATE, utilizzato nelle compressioni ZIP, GZIP e PNG, combina la codifica basata su dizionari di LZ77 con la codifica di Huffman per comprimere i puntatori delle stringhe ricorrenti. Lo svantaggio principale è che eredita tutti i limiti della codifica di Huffman (non ottimalità per entropie basse e per simboli correlati). Teoricamente sarebbe possibile sostituire la codifica di Huffman con la codifica aritmetica, ma si avrebbero notevoli costi computazionali e di licenze.

Algoritmo ANS

L'algoritmo ANS (Asymmetric Numeral Systems) si basa sul principio di funzionamento della codifica aritmetica, ma al posto di suddividere l'intervallo $[0, 1)$ in sottointervalli sempre più piccoli, si lavora con un insieme di numeri interi che man mano crescono in grandezza ogni volta che si espande la dimensione del blocco di simboli da codificare secondo la legge $X' \approx X/P(s)$. In questo modo si evita di dover lavorare con numeri floating point ad altissima precisione. Inoltre Jaroslaw Duda (l'ideatore dell'algoritmo ANS) ha rilasciato l'algoritmo in pubblico dominio senza brevetti contribuendo ad una rapida diffusione e integrazione in molti software di compressione.

Esistono due varianti dell'algoritmo ANS:

- **rANS (range ANS)**: le operazioni di aggiornamento degli stati interni vengono eseguite attraverso operazioni matematiche di moltiplicazioni e divisioni intere, risulta altamente parallelizzabile e permette di elaborare flussi di dati multipli; è usato in JPEG-XL e in Unreal Engine
- **tANS (table ANS)**: le operazioni di aggiornamento degli stati interni vengono eseguite attraverso una matrice di transizione pre-calcolata, risulta più veloce di rANS in quanto non ci sono operazioni matematiche, ma risulta meno parallelizzabile; è usato in Zstd di Meta, LZFS di Apple

5.5 Codifica Exp-Golomb

Struttura base

La codifica Exp-Golomb consiste nel codificare numeri interi con parole di codice di lunghezza crescente in funzione del modulo del numero da codificare. Siccome non tiene conto della distribuzione di probabilità dei simboli, non è ottimale, ma è molto semplice da implementare e risulta molto efficace per sorgenti in cui i simboli più probabili sono quelli a valori bassi (es. per dati sparsi come i coefficienti DCT di un'immagine o di un video).

Exp-Golomb applicata a numeri naturali - Unsigned EGC

Per codificare un numero naturale n con la codifica Exp-Golomb, si rappresenta il numero $n + 1$ in binario utilizzando $b = \lfloor \log_2(n + 1) \rfloor + 1$ bit e si aggiunge un prefisso formato da una sequenza di 0 (leading zeros) di lunghezza $b - 1$. La codifica di $n = 0$ è 1. Siccome la rappresentazione binaria di $n + 1$ inizia sempre con 1, la sequenza di leading zeros non può venire confusa con la restante parte della codeword, inoltre suggerisce il valore b facilitando la decodifica della codeword.

Ad esempio per il numero $n = 8$, la codifica binaria di $n + 1 = 9$ è 1001, la lunghezza della sequenza di leading zeros è $b - 1 = 3$, per cui la codeword finale è 0001001.

Exp-Golomb applicata a numeri interi - Signed EGC

Per codificare un numero intero n con la codifica Exp-Golomb, si utilizza una mappatura $m(n)$ dei numeri interi sui numeri naturali per riutilizzare la codifica unsigned EGC.

$$m(n) = \begin{cases} 2n - 1 & \text{se } n > 0 \\ -2n & \text{se } n \leq 0 \end{cases}$$

Ad esempio per il numero $n = 8$, la mappatura è $m(8) = 15$, la codifica binaria di $m(8) + 1 = 16$ è 10000, la lunghezza della sequenza di leading zeros è $b - 1 = 4$, per cui la codeword finale è 000010000.

5.6 Codifica per categoria e ampiezza

La codifica per categoria e ampiezza è simile a Exp-Golomb, ma ha lunghezze di codeword più corte per numeri elevati al costo di codeword più lunghe per numeri bassi.

Per codificare un numero intero n si utilizza un prefisso dato dal codice associato alla categoria k del numero calcolata come $k = \lceil \log_2(|n| + 1) \rceil$ e un suffisso dato dalla rappresentazione binaria di $|n|$ con k bit eventualmente in complemento a uno se $n < 0$ (negazione bit-a-bit).

Ad esempio per il numero $n = 8$, la categoria è $k = 4$ associata al codice 101, l'ampiezza è 1000, per cui la codeword finale è 1011000.

5.7 Codifica lossless predittiva

Processo di codifica

La codifica lossless predittiva consiste nell'utilizzare un predittore lineare per stimare il valore dei vari simboli e salvare/trasmettere solo i residui di predizione codificati con un codificatore lossless. Essendo i residui meno sparsi dei simboli originali, la codifica lossless dei residui risulta più efficiente (ha entropia più bassa) rispetto alla codifica diretta dei simboli originali. Inoltre il residuo è un segnale sparso con valori più probabili attorno allo zero, per cui la codifica Exp-Golomb risulta estremamente efficace e comparabile con la codifica ottima di Huffman.

I parametri utilizzati dal predittore lineare possono essere costanti per tutto il file o flusso, possono essere aggiornati dinamicamente in base ai residui calcolati o possono essere calcolati per minimizzare l'errore quadratico medio dei residui dei blocchi di simboli successivi.

Predittore semplice 1D

Il predittore semplice assume che il simbolo corrente x_i sia uguale al simbolo precedente x_{i-1} . Risulta vantaggio per segnali semi-stazionari o con variazioni lente. Nelle immagini equivale ad una predizione lungo la direzione orizzontale, ovvero risulta efficace per predire strutture orizzontali, ma non è efficace per predire strutture verticali o diagonali.

$$\hat{X}(n) = X(n-1)$$

Predittore avanzato 2D

Il predittore 2D, applicato alle immagini, assume che il pixel corrente $x_{i,j}$ sia uguale al pixel adiacente a sinistra $x_{i,j-1}$ o al pixel adiacente sopra $x_{i-1,j}$ in base a quale coppia di pixel adiacenti ha il valore più simile al pixel lungo la diagonale $x_{i-1,j-1}$. È capace di predire sia strutture orizzontali che verticali.

$$\hat{X}(i,j) = \begin{cases} x_{i,j-1} & \text{se } |x_{i,j} - x_{i,j-1}| \leq |x_{i,j} - x_{i-1,j}| \\ x_{i-1,j} & \text{altrimenti} \end{cases}$$

Applicazioni

Tale codifica è molto efficace per sorgenti in cui i simboli consecutivi sono fortemente correlati ad esempio per pixel adiacenti di un'immagine o per fotogrammi di un video.

Ad esempio un'immagine in scala di grigi con pixel a 8 bit, generalmente ha un'entropia di intorno ai 7 bpp (bit per pixel) in quanto i livelli di grigio sono distribuiti abbastanza uniformemente. Un predittore 1D riesce a ridurre l'entropia del residuo a circa 3.3 bpp e con un predittore 2D si riesce a ridurre ulteriormente fino a circa 2.8 bpp.

5.8 Codifica lossless tramite reti neurali

Processo di codifica

La codifica lossless tramite reti neurali si basa sull'addestramento di reti neurali (RNN, CNN, Transformers) come predittori dei simboli emessi dalla sorgente. In questo modo si riescono a catturare correlazioni più complesse su contesti di simboli più ampi e ottenere una distribuzione di probabilità altamente precisa per ogni blocco di simboli. Tale distribuzione viene poi codificata tramite codifica entropica (aritmetica o ANS) e salvata come residuo di predizione.

Vantaggi e svantaggi

Attraverso la predizione tramite reti neurali si riescono a raggiungere tassi di compressione molto vicini al limite teorico del tasso entropico della sorgente, inoltre si riescono a catturare dipendenze non lineari e pattern a lungo raggio. Gli svantaggi principali sono l'elevata complessità computazionale data dalle operazioni in floating point per l'inferenza della rete neurale che richiedono hardware specializzato (GPU, TPU) e tempi di compressione e decompressione più lunghi rispetto alle codifiche tradizionali.

6 Codifica JPEG

6.1 Processo di codifica JPEG



La codifica JPEG processa immagini in formato YCbCr (o in B/N), per cui il primo passo di codifica rimane la conversione da RGB a YCbCr con relativo sottocampionamento come visto in precedenza.

La codifica JPEG lavora solo su un canale, per cui in caso di immagini multicanale (YCbCr) ogni canale viene codificato separatamente. Il processo di codifica per un canale è composto da cinque fasi:

1. centratura dei valori dei campioni dei pixel di un canale: si sottrae il valore 128 a tutti i campioni dei pixel di un canale, in modo da portare i valori dall'intervallo $[0, 255]$ all'intervallo $[-128, 127]$ per far lavorare meglio la DCT
2. partizione dell'immagine in blocchi di dimensione 8×8 pixel: la dimensione è stata scelta come compromesso tra complessità computazionale (blocchi grandi richiedono più risorse) e la qualità della compressione (blocchi piccoli non catturano strutture globali) in base ai calcolatori e alle risoluzioni diffusi quanto è stato definito lo standard (1991)
3. calcolo dei coefficienti della matrice DCT per ogni blocco (approfondito in seguito)
4. quantizzazione dei coefficienti DCT (approfondito in seguito)
5. codifica lossless dei coefficienti quantizzati (approfondito in seguito)

Lo standard definisce la sintassi del formato, ma non pone vincoli sugli algoritmi implementativi, permettendo lo stesso la concorrenza tra diversi produttori di codec.

6.2 Discrete Cosine Transform - DCT

Trasformate sparsificanti per generare residui a bassa entropia

L'obiettivo di una codifica lossy è quello di eseguire una nuova quantizzazione dei dati per comprimere l'informazione memorizzata. Il problema è che se si esegue una riquantizzazione diretta sui campioni, la qualità del segnale digitale degrada molto velocemente in quanto i campioni non sono sparsi e hanno tutti approssimativamente la stessa importanza. È possibile, però, applicare trasformate lineari sparsificanti che rappresentano il segnale in un nuovo dominio in cui i campioni sono più sparsi e su cui è possibile eseguire una quantizzazione più aggressiva per eliminare i campioni meno importanti senza degradare troppo la qualità del segnale. La trasformata deve essere anche facilmente invertibile per poter tornare al dominio dei campioni originali.

Trasformate ortogonali preservano le metriche di prestazione

Tra tutte le trasformate sparsificanti, si preferiscono quelle ortogonali per cui detta A la matrice di trasformazione si ha che $A^{-1} = A^T$. La proprietà di queste trasformazioni è che preservano la distanza euclidea tra due segnali, e di conseguenza anche le metriche di prestazione come il MSE e il PSNR. Per cui una strategia di compressione ottimale (che massimizza il PSNR nella nuova rappresentazione) è anche ottimale nel dominio dei campioni originali.

Di seguito si illustra il processo di trasformazione (1), quantizzazione (2) e ricostruzione (3) del segnale e si dimostra che l'MSE si conserva tra il dominio dei campioni e quello dei coefficienti trasformati

0. $\vec{x} = [s(n+1), s(n+2) \dots s(n+N)]^T \in \mathbb{R}^N$ segnale originale in notazione vettoriale
1. $\vec{y} = A \vec{x}$ applicazione della trasformata ortogonale
2. $\tilde{y} = [Q(y(1), \Delta_1), Q(y(2), \Delta_2), \dots, Q(y(N), \Delta_N)]$ quantizzazione dei coefficienti trasformati
3. $\tilde{x} = A^{-1} \tilde{y}$ ricostruzione del segnale originale

$$\begin{aligned}
 \text{MSE}(\vec{x}, \tilde{x}) &= \frac{1}{N} \|\vec{x} - \tilde{x}\|^2 = \frac{1}{N} (\vec{x} - \tilde{x})^T \cdot (\vec{x} - \tilde{x}) = \frac{1}{N} (A^T \vec{y} - A^T \tilde{y})^T \cdot (A^T \vec{y} - A^T \tilde{y}) = \\
 &= \frac{1}{N} (\vec{y} - \tilde{y})^T \cdot A \cdot A^T \cdot (\vec{y} - \tilde{y}) = \frac{1}{N} \|\vec{y} - \tilde{y}\|^2 = \text{MSE}(\vec{y}, \tilde{y})
 \end{aligned}$$

Trasformate bidimensionali applicate alle immagini

Le trasformate ortogonali sono state pensate per lavorare su vettori, per cui per applicarle alle immagini (matrici) è necessario prima applicarle alle righe e poi alle colonne della matrice dei campioni come segue. Si nota applicando la trasformata prima sulle righe e poi sulle colonne o viceversa, il risultato non cambia.

$$Y = A X A^T = \underbrace{(A X)}_{\text{trasformata delle colonne}} A^T = A \underbrace{(A X^T)^T}_{\text{trasformata sulle righe}}$$

- $Z = A X$ trasformata delle colonne, il risultato è una matrice le cui colonne corrispondono al prodotto tra la matrice di trasformazione A e le colonne di X
- $Y = Z A^T = (A X^T)^T$ trasformata sulle righe, il risultato è una matrice le cui righe corrispondono al prodotto tra la matrice di trasformazione A e le righe di Z

Trasformata di Fourier e Discrete Cosine Transform applicata alle immagini

Si nota che la trasformata di Fourier è una buona candidata per essere una trasformata sparsificante e ortogonale. In pratica rappresenta l'immagine come combinazione di sinusoidi orizzontali e verticali. Le basse frequenze corrispondono a variazioni lente del colore dei pixel (ad esempio superfici uniformi), mentre le alte frequenze corrispondono a variazioni rapide (ad esempio zone ricche di dettagli). Siccome le immagini in generale sono composte principalmente da basse frequenze, la trasformata di Fourier riesce a sparsificare bene i dati.

Nella compressione JPEG, però, si usa la Discrete Cosine Transform (DCT) che è una variante discreta della trasformata di Fourier basata solo sulle funzioni coseno, restituisce solo coefficienti reali positivi e permette una migliore sparsificazione.

Interpretazione della DCT bidimensionale a blocchi

Nell'algoritmo JPEG, si applica la DCT su blocchi di immagine di dimensione 8×8 pixel, ottenendo una matrice di coefficienti DCT di dimensione 8×8 per ogni blocco. La scelta della dimensione del blocco è un compromesso tra complessità e qualità della compressione.

Ogni coefficiente della matrice DCT rappresenta la correlazione (o prodotto interno) tra il blocco di pixel dell'immagine e un particolare blocco della DCT costruito ad hoc per rappresentare un particolare pattern di frequenza. Essendo i blocchi della DCT ordinati in frequenza crescente, le matrici di coefficienti avranno una struttura sparsificata concentrando i coefficienti associati alle basse frequenze in alto a sinistra e quelli associati alle alte frequenze in basso a destra.

Corrispondenza tra coefficienti DCT e le strutture dell'immagine

Analizzando la matrice di coefficienti DCT di un'immagine si notano le seguenti strutture:

- superfici uniformi fanno crescere i coefficienti a bassa frequenza, concentrati in alto a sinistra
- i dettagli fanno crescere i coefficienti a medio/alte frequenze, concentrati al centro
- bordi e contorni fanno crescere i coefficienti lungo rette passanti per l'origine in alto a sinistra
- strutture periodiche dell'immagine formano punti isolati di medio/alta frequenza, corrispondenti proprio alla frequenza della struttura periodica

6.3 Quantizzazione dei coefficienti DCT

Quantizzazione dei coefficienti DCT

La quantizzazione dei coefficienti DCT avviene tramite un quantizzatore uniforme e converte i coefficienti reali della DCT in valori interi detti indici di quantizzazione. Tali indici esprimono il fattore moltiplicativo dell'intervallo di quantizzazione Δ necessario per ottenere il coefficiente DCT quantizzato. In questo modo si riesce a ridurre i valori da memorizzare e rendere la codifica lossless successiva più efficiente.

$$x_{\text{ind}} = Q(x, \Delta) = \text{round} \left(\frac{x}{\Delta} \right) \quad \tilde{x} = \Delta \cdot x_{\text{ind}} = \Delta \cdot \text{round} \left(\frac{x}{\Delta} \right)$$

Tabella di quantizzazione

Siccome i coefficienti della DCT hanno importanza diversa in base alla loro frequenza, è possibile utilizzare quantizzatori diversi specifici per ogni coefficiente della matrice DCT. Si definisce, quindi, una tabella di quantizzazione q di dimensione 8×8 che contiene i valori di quantizzazione $\Delta(i, j)$ specifici per i corrispettivi coefficienti $Y(i, j)$ della matrice DCT.

Lo standard non fornisce una tabella specifica, ma ne suggerisce una di riferimento per la luminanza e una per la crominanza ottenute tramite test soggettivi sulla qualità percepita. Tali matrici hanno valori più bassi associati alle basse frequenze in alto a sinistra e valori più alti associati alle alte frequenze in basso a destra.

Fattore di qualità

Per rendere la codifica più flessibile all'utente, si definisce un fattore di qualità Q compreso tra 1 e 100 che definisce il fattore di scala SF per la tabella di quantizzazione base q^* . In questo modo l'utente può scegliere se utilizzare compressioni più aggressive (Q basso) o più conservative (Q alto).

Siccome la tabella deve essere costituita da interi nel range $[1, 256]$ per essere rappresentata in 64 byte (1 byte per valore), a seguito dello scalamento per SF viene eseguita l'operazione di troncamento della parte decimale e se il valore è maggiore di 256, viene sostituito con 256.

$$SF = \begin{cases} 5000/Q & \text{se } 0 < Q \leq 50 \\ 200 - 2Q & \text{se } 50 < Q \leq 99 \\ 1 & \text{se } Q = 100 \end{cases} \quad q = \frac{SF}{100} q^*$$

Effetti della quantizzazione delle frequenze

Per immagini con bassa qualità, spesso si verificano fenomeni di sdoppiamento dei bordi e dei contorni dell'immagine. Questo è dovuto al fatto che comprimendo pesantemente le alte frequenze, queste non riescono più a delimitare correttamente i bordi e le zone di alta frequenza e le oscillazioni lente si propagano anche nei pixel adiacenti. Si parla di propagazione delle strutture a bassa frequenza.

Per fattori di compressione molto bassi, si possono notare le strutture a blocchi di 8×8 pixel.

6.4 Codifica lossless dei coefficienti DCT

Zig-zag scan

Per convertire la matrice di coefficienti DCT in un vettore, si utilizza lo zig-zag scan che parte dal coefficiente in alto a sinistra e si prosegue per diagonali ordinando i coefficienti in frequenza crescente. Al posto di riportare tutta la sequenza di coefficienti nulli alla fine del vettore, si posiziona un simbolo speciale EOB (End Of Block) dopo l'ultimo coefficiente non nullo.

Codifica del coefficiente DC

Il primo coefficiente del vettore ottenuto dallo zig-zag scan è detto coefficiente DC e rappresenta la componente continua di frequenza nulla. Per codificarlo si utilizza una codifica predittiva usando il valore DC del blocco precedente e il residuo viene codificato con codifica categoria/valore. La tabella di codifica dei residui DC non è definita dallo standard per cui va indicata nel file JPEG.

$$e_{\text{residuo}} = x_{\text{DC del blocco corrente}} - p_{\text{DC del blocco precedente}}$$

Codifica dei blocchi AC

I restanti coefficienti del vettore sono detti AC e rappresentano le componenti di frequenza non nulla. Per codificarli si utilizza la codifica run-length che consiste nel creare una sequenza di simboli (R, C) in cui R è il numero di coefficienti nulli consecutivi e C è il primo coefficiente AC non nullo. Se si incontrano più di 15 coefficienti nulli, si inserisce un simbolo speciale (15, 0) e il successivo simbolo calcola R a partire dal successivo coefficiente. La sequenza termina con il simbolo (0, 0). La codifica dei simboli (R, C) avviene in due fasi: prima si codifica C con codifica categoria/valore (k, v) , poi si codifica la coppia (R, k) con una codifica a prefisso. La tabella di codifica dei blocchi AC non è definita dallo standard per cui va indicata nel file JPEG.

6.5 Sintassi del formato JPEG baseline (1992)

Struttura del file JPEG

Un file JPEG è costituito da una struttura gerarchica di segmenti, come segue:

- start of image: sezione di apertura del file
- frame header: informazioni sulla dimensione dell'immagine, sullo spazio colore, sul campionamento dei canali
- frame: sezione suddivisa a sua volta in uno o più scan, uno per ogni canale di cui è composta l'immagine, ognuno dei quali contiene:
 - scan header: informazioni sul canale e sulle tabelle di quantizzazione
 - scan: sezione costituita da una sequenza di segmenti che possono essere codificati e decodificati in parallelo, ognuno dei quali contiene:
 - segment header: informazioni sulle tabelle di codifica
 - segment: dati effettivi codificati, suddivisi in blocchi della DCT
- end of image: sezione di chiusura del file

6.6 Evoluzione dei formati di compressione delle immagini

- **1992 - JPEG baseline:**
primo standard JPEG (8×8 , DCT, zig-zag scan, Huffman), ampiamente usato e supportato da tutti i browser e sistemi operativi per la sua semplicità e compatibilità
- **1996 - PNG:**
formato lossless con predizione spaziale e codifica con dizionario con Huffman, usato per immagini nel web e per immagini con trasparenza, poco diffuso in smartphone e fotocamere
- **2000 - JPEG 2000:**
viene sostituita la DCT con la DWT (Discrete Wavelet Transform) più adatta a rappresentare le immagini, permette una codifica progressiva lossy-to-lossless, usato nel cinema digitale, imaging medico e immagini satellitare
- **2010 - WebP:**
proveniente dai codec video, basato su predizione spaziale, DCT a dimensione variabile e deblocking filter, usato per immagini nel web
- **2010 - WebP lossless:**
proveniente dai codec video, versione avanzata della PNG
- **2015 - HEIC:**
proveniente dai codec video, versione avanzata di WebP, usata nell'ecosistema Apple, molto efficiente, ma soggetta a royalty su tecnologie brevettate (solo aziende grosse come Apple si possono permettere di gestire la burocrazia per ottenere le licenze)
- **2019 - AVIF:**
versione di HEIC e WebP senza royalty, molto diffusa in ambito web e smartphone come alternativa efficiente a JPEG
- **2019 - JPEG XL:**
versione avanzata di JPEG 2000, con codifica progressiva lossy-to-lossless, aggiunge codifica entropica dell'intero file, supporto per immagini HDR e migliore gestione dei bordi e texture ad alte frequenze, dovrebbe diventare il nuovo standard JPEG, ma il suo supporto è ancora limitato
- **2025 - JPEG AI:**
sfrutta le codifiche basate su reti neurali (DNN e autoencoder) che utilizzano approcci di deep learning per estrapolare le caratteristiche principali dell'immagine e codificarle in maniera efficiente in uno spazio latente, riescono a modellare meglio strutture non lineari riducendo parecchio il bitrate, esistono due varianti: una ad alta complessità (richiede GPU e NPU) ed una a bassa complessità (sufficiente una CPU, per smartphone), potrebbe diventare il futuro standard JPEG

7 Codifica video

7.1 Principi di compressione video

Predizione temporale e decoded frame buffer

I codificatori video, oltre alla compressione spaziale e alla compressione basata su JPEG, sfruttano anche la ridondanza temporale tra fotogrammi successivi attraverso predizioni temporali. Le immagini consecutive di un video, infatti, sono molto simili tra loro e tale sistema di predizione risulta estremamente efficace per ridurre il bitrate.

In pratica il codificatore calcola i parametri del modello e l'errore di predizione dell'immagine corrente in funzione di una o più immagini precedenti (o successive) e invia al decodificatore tali informazioni. Il decodificatore avrà un buffer di immagini decodificate (decoded frame buffer) dove potrà memorizzare temporaneamente le immagini decodificate per poterle usare come riferimento per la predizione dei fotogrammi successivi.

Criticità della predizione temporale e GOP

Con la predizione temporale si forma una catena continua di dipendenze temporali tra i fotogrammi e si verificano due criticità principali:

- propagazione di errori: se un fotogramma viene perso o danneggiato, tutti i fotogrammi successivi che dipendono da esso saranno anch'essi danneggiati
- mancanza di accesso casuale: per accedere a un fotogramma intermedio, è necessario decodificare tutti i fotogrammi precedenti in un processo lungo e inutilmente costoso.

Per mitigare tali problemi, i fotogrammi vengono raggruppati in unità chiamate GOP (Group of Pictures) che rendono fotogrammi di GOP diversi indipendenti tra loro.

Parametri e metriche di prestazione dei codificatori video

I parametri di progetto che caratterizzano un codificatore video sono:

- modalità di predizione temporale per la stima del movimento
- struttura del GOP e dipendenze tra fotogrammi
- strategia di mode selection per la predizione spaziale e temporale

Gli indici di prestazione di un codificatore video sono:

- bitrate: quantità di dati per unità di tempo necessaria per trasmettere il video
- qualità dell'immagine: misurata in termini di PSNR o tramite test soggettivi
- complessità computazionale: tempo di codifica e decodifica e risorse computazionali richieste
- ritardo di codifica e decodifica: tempo necessario per accumulare dati prima di poterli codificare o decodificare (es. per la predizione temporale rispetto ad immagini successive)
- robustezza agli errori: capacità di resistere a errori di trasmissione e perdita di pacchetti

7.2 Stima e compensazione del movimento per block matching

Suddivisione in blocchi

Il movimento in un video può essere dovuto allo spostamento degli oggetti nella scena o al movimento della telecamera. Per stimare correttamente il movimento dei singoli oggetti, l'immagine viene suddivisa in blocchi di dimensione $N \times M$ pixel e per ciascuno viene fatta la predizione del movimento. I blocchi si indicano attraverso la notazione $B_k^{(\vec{p})}$ in cui k è l'indice del fotogramma e $\vec{p}(x, y)$ è la posizione del pixel centrale del blocco.

Stima del movimento con vettore e compensazione

L'obiettivo della stima del movimento non è quello di tracciare il movimento degli oggetti, ma di trovare un blocco di riferimento più simile a quello da predire per minimizzare il residuo di predizione.

La stima del movimento consiste nell'individuare il vettore di movimento $\vec{v}(x, y)$ che collega il blocco da predire $B_k^{(\vec{p})}$ ad un altro blocco $B_h^{(\vec{p}+\vec{v})}$ di un altro fotogramma h usato come riferimento per la predizione tale da minimizzare il costo di predizione $J(\vec{v})$.

$$J(\vec{v}) = d(B_k^{(\vec{p})}, B_h^{(\vec{p}+\vec{v})}) + \lambda R(\vec{v}) \quad \vec{v}^* = \arg \min_{\vec{v} \in \mathcal{W}} J(\vec{v}^*)$$

La dissimilarità (o differenza) tra due blocchi $d(\vec{v}) = d(B_k^{(\vec{p})}, B_h^{(\vec{p}+\vec{v})})$ in funzione del vettore di movimento può essere calcolata come somma delle differenze assolute dei pixel (SAD) o come somma dei quadrati delle differenze dei pixel (SSD).

Siccome il costo complessivo è dato sia dalla rappresentazione del residuo che da quella del vettore $\vec{v}$, la formula di costo $J(\vec{v})$ include un termine di penalizzazione $R(\vec{v})$ che rappresenta il costo per rappresentare il vettore di movimento e un parametro λ che bilancia l'importanza dei due termini.

Algoritmi e pattern di ricerca del vettore di movimento

L'algoritmo più semplice per la ricerca del vettore di movimento è la ricerca esaustiva (full search) che analizza il costo di predizione per tutti i possibili blocchi nella finestra di ricerca $\mathcal{W}$ e seleziona quello con il costo minimo. Tale algoritmo è ottimale, ma molto complesso e costoso in termini di tempo di calcolo.

In alternativa esistono algoritmi euristici subottimali come la ricerca hexagon che espande la ricerca analizzando i blocchi adiacenti e proseguendo solo nella direzione del blocco con il costo minore. La complessità computazionale è molto più bassa, ma non viene garantita l'ottimalità della soluzione.

Codifica del residuo di predizione

La codifica del residuo di predizione può avvenire tramite diverse codifiche in base al contesto:

- codifica Exp-Golomb, efficace in generale siccome i residui hanno valori sparsi e vicini allo zero
- codifica aritmetica, più efficace se c'è poco movimento e quindi i residui sono molto vicini allo zero
- codifica JPEG semplificata, in generale siccome i residui hanno valori molto simili tra di loro e per molti blocchi è sufficiente solo il coefficiente DC
- codifica JPEG completa, se il residuo ha valori poco sparsificati ad esempio per cambi di scena

Parametri e metriche prestazione della ricerca block-matching

I parametri di progetto che caratterizzano la stima del movimento sono:

- forma e dimensione dei blocchi: blocchi piccoli riducono il costo dei residui di predizione, ma aumentano il numero di vettori di movimento da rappresentare e quindi anche costo complessivo
- dimensione della finestra di ricerca $\mathcal{W}$: aumentando la finestra di ricerca aumenta la probabilità di trovare blocchi di riferimento più simili, ma aumenta anche la complessità computazionale
- funzione di dissimilarità: si preferisce la SAD in quanto più semplice da calcolare e simile al MSE
- strategia di ricerca: la ricerca esaustiva è la più accurata, ma anche la più complessa, per cui si preferiscono algoritmi euristici subottimali per ridurre la complessità computazionale

Le prestazioni della stima del movimento si misurano in termini di:

- accuratezza della stima: misurata in termini di PSNR o MSE del residuo di predizione
- bitrate o costo della predizione: misurato in termini di numero di bit necessari per rappresentare il residuo e il vettore di movimento
- complessità computazionale: misurata in termini di tempo di calcolo e risorse computazionali richieste

Criticità della stima del movimento

La stima del movimento risulta poco efficace in prossimità dei bordi dell'immagine o in aree di disocclusione in cui compaiono nuovi oggetti che non hanno corrispondenza nei fotogrammi precedenti.

Si nota inoltre che in aree senza strutture come il cielo o superfici uniformi, il block-matching si basa sui pattern di rumore del blocco, che non sempre sono coerenti con il movimento reale. Tale problema, risulta irrilevante siccome il residuo di predizione sarà comunque quasi nullo.

7.3 Tipi di immagine e GOP

Tipi di immagine/frame

Le immagini o frame di un group of pictures si classificano in tre categorie principali:

- frame I - intra coded: frame codificati senza predizione temporale, ma solo con predizione spaziale dalla stessa immagine o codifica JPEG per cui hanno bassa complessità di codifica/decodifica; sono usati come base per la predizione di tutti gli altri frame del GOP e garantiscono l'accesso casuale rapido al video e maggiore immunità agli errori
- frame P - predictive: frame codificati con predizione temporale da un frame precedente (I o P) oppure tramite predizione spaziale dalla stessa immagine, risultano di complessità maggiore rispetto ai frame I, ma hanno bitrate inferiore
- frame B - bidirectional predictive: frame codificati con predizione temporale da due frame, uno precedente e uno successivo (I o P), in questo modo si aumenta la probabilità di trovare blocchi di riferimento più simili a discapito di una maggiore complessità di codifica/decodifica e all'introduzione di latenza in quanto dipendono da frame successivi

I frame I e P sono detti anchor frame (AF) in quanto sono usati come riferimento per la predizione di altri frame (P o B).

Struttura di un Group-Of-Pictures

Il GOP è una sequenza di immagini che inizia con un frame I e termina con il frame precedente al successivo frame I. Un GOP è definito da due parametri principali:

- N : intervallo da un frame I al successivo frame I, denota il numero di frame nel GOP
- M : intervallo tra due AF successivi, denota il numero di frame B tra due I o P successivi

Tasso e distorsione dei vari tipi di immagine

Si osserva che i fotogrammi I hanno un bitrate di 3-5 volte maggiore rispetto ai fotogrammi P e 10-20 volte maggiore rispetto ai fotogrammi B. La qualità dei fotogrammi I è forzosamente maggiore rispetto ai fotogrammi P e B in quanto sono usati come riferimento per l'intero GOP.

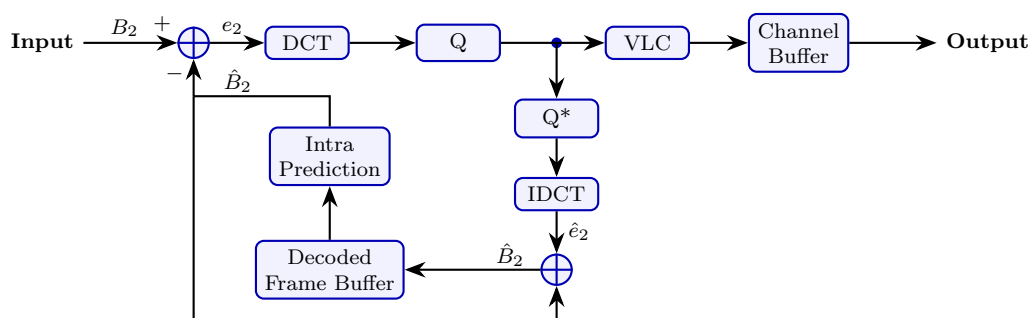
I parametri N e M del GOP fungono da compromesso tra il bitrate complessivo e la complessità di codifica/decodifica. Avere tanti AF aumenta il bitrate complessivo, ma riduce la complessità di codifica/decodifica, viceversa aumentando il numero di frame B si riduce il bitrate, ma si aumenta la complessità di codifica/decodifica e la latenza.

7.4 Codificatori e decodificatori ibridi

Codificatore ibrido per frame intra

Il codificatore ibrido per frame intra è composto da:

- catena di blocchi per la codifica simil-JPEG (DCT, quantizzazione Q e variable length coding VLC)
- anello di loopback che ricostruisce il frame decodificato tramite la normalizzazione dei coefficienti quantizzati (Q^*) e la Inverse DCT tipici del decodificatore JPEG ed effettua una predizione spaziale intra-frame tra i blocchi del frame corrente per ridurre la ridondanza spaziale tra blocchi adiacenti



Codificatore ibrido per frame inter

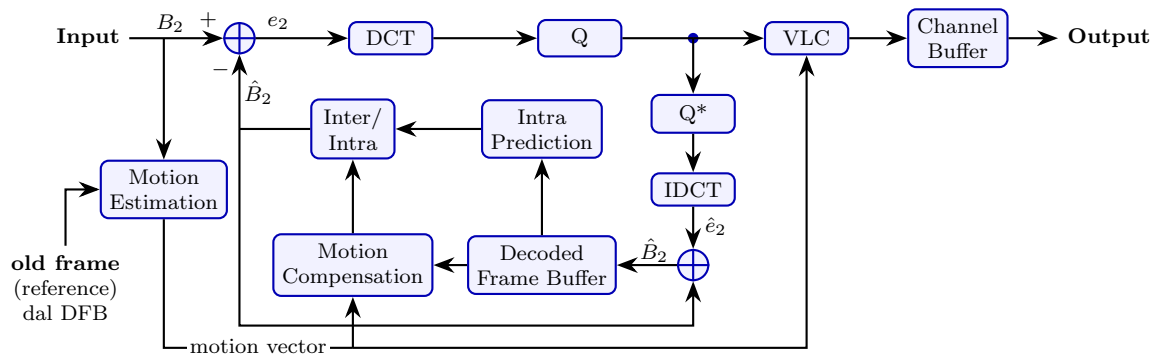
Il codificatore ibrido per frame inter è composto da:

- la stessa base del codificatore di tipo intra con codifica simil-JPEG e predizione spaziale intra-frame
- un blocco di motion estimation (ME) che calcola il vettore di movimento $\vec{v}$
- un blocco di motion compensation (MC) che calcola il blocco predetto $\hat{B}_k^{(\vec{v})}$ a partire dal vecchio frame di riferimento nel decoded frame buffer e dal vettore di movimento $\vec{v}$
- un blocco di selezione tra predizione spaziale di tipo intra e predizione temporale di tipo inter

La scelta tra l'utilizzo della predizione spaziale o quella temporale dipende da un criterio di valutazione del compromesso tra bitrate e distorsione come il seguente:

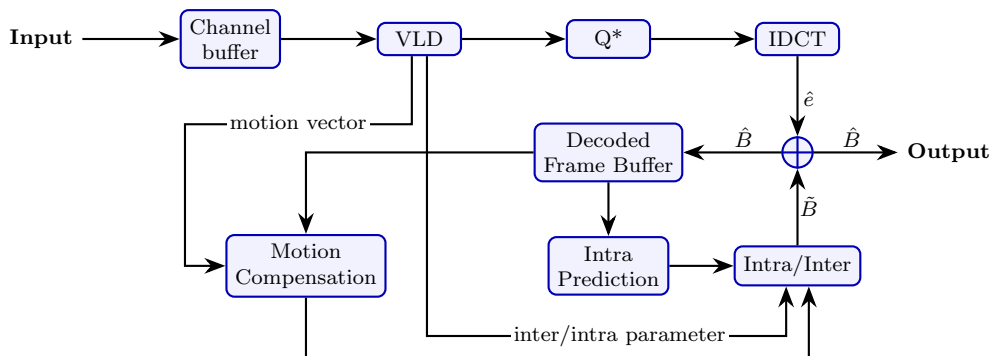
$$m_{\text{modo di codifica}} \in \{\text{INTRA}, \text{INTER}\} \quad m^* = \arg \min_m \underbrace{D_m}_{\text{distorsione}} + \lambda \underbrace{R_m}_{\text{bitrate}}$$

Lo standard non definisce il tipo di criterio da utilizzare, la scelta che viene lasciata al progettista del codificatore che potrebbe anche pensare di usare sempre solo una delle due predizioni per risparmiare tempo di calcolo. In questo modo si permette di avere concorrenza tra diversi produttori.



Decodificatore ibrido

Il decodificatore ibrido ripercorre al contrario la catena di elaborazione del codificatore ibrido. Risulterà più veloce rispetto ai codificatori quanto non deve effettuare calcoli sulla predizione del movimento o selezioni tra predizione spaziale e temporale.



7.5 Standard di codifica video principali

Evoluzione degli standard di codifica video

- **1995 - MPEG-2:**
primo standard video di grande diffusione usato per DVD e TV satellitare, è composto da un codec ibrido (I,P,B) senza predizione spaziale (è presente solo JPEG), supporta fino a 1080p a 60fps con bitrate di 80 Mbps, fornisce strumenti di codifica scalabile ovvero è possibile estrarre più flussi video con qualità ridotte da un unico flusso codificato (utile per adattare la trasmissione alla rete)
- **2003 - H.264/AVC:**
standard ancora oggi più diffuso e usato per lo streaming, rispetto a MPEG-2 permette predizione spaziale, blocchi a dimensione variabile, trasformate simil-DCT intere e viene aggiunto un deblocking filter a fine codifica per rimuovere gli artefatti di quantizzazione, il bitrate viene dimezzato rispetto a MPEG-2
- **2013 - H.265/HEVC:**
usato per streaming 4K e DVB-T2, è composto da un codec ibrido basato su alberi quaternari, il bitrate viene dimezzato rispetto a H.264, non è molto diffuso perché protetto da licenze e royalties
- **2018 - AV1:**
alternativa royalty-free al H.265, usato per streaming Youtube e Netflix
- **2021 - VVC/H.266:**
usato per video 8K e VR, è composto da un versatile video coding che aumenta la alta complessità di 10× rispetto a H.265, dimezzandone il bitrate
- **2025+ - Neural Video Coding:**
nuovi codec basati su reti neurali (DNN e autoencoders) contenuti nei nuovi standard MPEG-AI

Digitale terrestre

- **H.265/HEVC** 55-60% in crescita: standard per il DVB-T2 in Europa (diventato standard nell'ottobre 2025) e ATSC 3.0 negli USA, necessario per le trasmissioni in 4K e HDR
- **H.264/AVC** 30-35% in calo: usato come fallback per i canali SD/HD o in televisori più vecchi
- **MPEG-2** 5-10% in calo: limitato a mercati di nicchia o canali locali

Streaming

- **AV1** 35-40%: adottato massicciamente da Netflix, Youtube e Meta
- **H.264/AVC** 30%: utilizzato per retrocompatibilità con dispositivi più vecchi
- **H.265/HEVC** 25%: per contenuti 4K HDR su dispositivi Apple e TV di fascia alta

Video real-time

- **H.264/AVC** 50%: molto usato grazie all'accelerazione hardware molto diffusa
- **VP9** 25%: versione royalty free di H.264, utilizzato da Google Meet e in applicazioni browser-based grazie alla sua efficienza superiore rispetto a H.264
- **VP8** 15%: ancora presente in WebRTC per la bassissima complessità computazionale
- **AV1** 10% in crescita: utilizzato per scenari di banda estremamente limitata, grazie al basso bitrate

8 Metriche di rete

8.1 Architettura di rete a strati

Concetti di livello, servizio, interfaccia e protocollo di un'architettura a strati

Un'architettura a strati è un modello concettuale che suddivide il funzionamento di un sistema in una pila gerarchica di livelli o strati. Ogni livello svolge una parte di compiti, necessari il funzionamento corretto dell'intero sistema. Permette di suddividere un problema complesso in più sotto-problemi più semplici da risolvere, favorendo la modularità, la riusabilità e la manutenibilità del sistema.

Nelle implementazioni reali si cerca di mantenere il più possibile indipendenti i livelli, in modo da garantire modularità e riusabilità del sistema, ma in alcuni casi si utilizzano implementazioni cross-layer per migliorare l'efficienza e le prestazioni del sistema.

Un'architettura a strati si basa su quattro concetti fondamentali:

- **livello:** insieme di componenti che risolvono una serie di sottoproblemi omogenei del problema complesso; ogni livello fornisce un insieme di servizi al livello superiore nascondendone i dettagli di implementazione
- **servizio:** insieme di primitive o funzionalità che un livello offre al livello superiore
- **interfaccia:** insieme di regole e convenzioni che definiscono come un livello può accedere ai servizi del livello sottostante
- **protocollo:** insieme di regole e convenzioni che definiscono come due livelli corrispondenti situati in nodi diversi della rete comunicano tra loro

Livelli nelle reti di calcolatori - modello ISO/OSI

Il modello ISO/OSI (Open Systems Interconnection) è il modello di riferimento per le reti di calcolatori, suddivide il funzionamento di una rete nei seguenti sette livelli:

0. **physical medium:** rappresenta il mezzo fisico di trasmissione di segnali analogici, come cavi, fibre ottiche, onde radio, ecc., collega fisicamente i vari nodi della rete
- L1. **physical layer:** fornisce un canale digitale di trasmissione a bit inaffidabile tra due nodi adiacenti della rete, gestisce la conversione digitale/analogica e viceversa per trasmettere i bit digitali sul mezzo fisico tramite segnali analogici
- L2. **data link:** fornisce un canale digitale di trasmissione a bit affidabile tra due nodi adiacenti della rete, gestisce la correzione di errori sul bit introdotti dal canale inaffidabile del livello data link
- L3. **network:** fornisce un canale digitale di trasmissione a pacchetti inaffidabile tra due nodi non necessariamente adiacenti della rete, gestisce l'instradamento dei pacchetti attraverso la rete (non garantisce che i pacchetti arrivino a destinazione, né che arrivino in ordine)
- L4. **transport:** fornisce un canale di comunicazione end-to-end tra due processi in esecuzione su due nodi diversi della rete, gestisce la segmentazione dei dati in pacchetti, la correzione di errori sul pacchetto e il controllo del flusso di trasmissione dei pacchetti (ad esempio l'avvenuta ricezione)
- L5. **session:** fornisce una connessione logica tra due processi in esecuzione su due nodi diversi della rete, gestisce la creazione e la distruzione di un canale di comunicazione tra due processi
- L6. **presentation:** fornisce un sistema di scambio dei dati tra due processi in esecuzione su due nodi diversi della rete, gestisce la codifica, compressione, cifratura e formattazione dei dati da trasmettere
- L7. **application:** fornisce un'interfaccia tra l'utente e la rete, gestisce gli input dell'utente e la visualizzazione dei dati ricevuti dalla rete

Incapsulamento dei dati e aggiunta degli headers

In fase di trasmissione, ogni livello di rete, riceve i dati dal livello superiore detti payload e, prima di trasmetterli al livello inferiore, ci aggiunge un header contenente informazioni di protocollo necessarie al livello corrispondente del nodo ricevente per poter interpretare correttamente i dati ricevuti. Il pacchetto formato da header e payload ad un certo livello diventa, quindi, il payload del livello inferiore.

Viceversa in fase di ricezione, ogni livello di rete, riceve i dati dal livello inferiore, separa l'header di livello dal payload, processa il payload con le informazioni dell'header appena estratto e passa i dati processati al livello superiore senza l'header di livello appena consumato.

8.2 Paradigma di trasmissione a pacchetto

Pacchettizzazione del livello 4 di transport

Il livello 4 di transport suddivide i dati da trasmettere, ricevuti dai livelli superiori, in pacchetti per poterli trasmettere agevolmente attraverso la rete. Teoricamente la pacchettizzazione dovrebbe essere invisibile ai livelli superiori però in alcuni casi è utile attuare un'implementazione cross-layer in cui i pacchetti coincidono le code unit dei codec video (blocchi in cui vengono divisi i frame).

Vantaggi della trasmissione a pacchetti

I vantaggi della trasmissione a pacchetti sono:

- **efficienza**: i pacchetti possono essere instradati in modo indipendente attraverso la rete, permettendo di sfruttare al meglio le risorse disponibili e ridurre i tempi di trasmissione
- **affidabilità**: in caso di errore o perdita di pacchetti, è possibile ritrasmettere solo i pacchetti persi o danneggiati, invece di dover ritrasmettere l'intero flusso di dati
- **scalabilità**: la trasmissione a pacchetti consente di gestire la comunicazione tra un numero elevato di nodi e dispositivi con grandi volumi di traffico senza cambiare l'architettura dell'infrastruttura
- **multiplexing statistico**: più flussi di dati possono condividere le stesse risorse di rete garantendo una maggiore flessibilità

8.3 Metriche di traffico

Unità di misura per il tasso di trasmissione

Il tasso di trasmissione o bitrate si può misurare in:

- bit/s (bps): bit per secondo, con relativi multipli kbps, Mbps, Gbps, Tbps
- byte/s (Bps): byte per secondo, con relativi multipli kBps, MBps, GBps, TBps, con $1 \text{ B} = 8 \text{ b}$

Per convenzione, non si utilizza la notazione binaria per cui $1 \text{ MB/s} = 2^{20} \text{ B/s}$, bensì si usa quella decimale in cui $1 \text{ MB/s} = 10^6 \text{ B/s}$.

Bitrate, throughput, goodput

Tutti e tre i termini indicano il tasso o velocità di trasmissione dei dati tra due entità peer (livelli corrispondenti) di due nodi della rete, ma differiscono per il livello in cui sono misurati:

- **bitrate**: velocità nominale del livello fisico L1, indicato con R_0 , dipende dalla larghezza di banda, dalla modulazione utilizzata e dal rapporto segnale/rumore del canale, è il tasso massimo di trasmissione dei bit su canale single-hop inaffidabile (canale tra nodi adiacenti senza garanzia di correttezza e consegna dei bit)
- **throughput**: velocità media misurata tra il livello 2 di data link e il livello 4 di transport, indicata con S , in base al livello in cui si misura può tenere conto dell'overhead dovuto agli error correction codes, agli header o alla ritrasmissione dei pacchetti persi o danneggiati
- **goodput**: velocità effettiva misurata in un livello superiore al 4 (session, presentation o application), corrisponde al throughput a lungo termine S_n e rappresenta il tasso di trasmissione dei dati utili, tenendo conto di tutti gli overhead necessari alla trasmissione dei dati

In generale si ha che $R_0 \geq S \geq \text{goodput}$, ovvero il tasso di trasmissione diminuisce con l'aumentare dei livelli di rete a causa dell'overhead introdotto dagli header di ogni livello.

Protocol Data Unit, Protocol Control Information e Service Data Unit

- **Protocol Data Unit (PDU)**: insieme di dati che viene trasmesso tra due livelli corrispondenti di due nodi della rete, coincide con il payload, ovvero i dati utili da trasmettere
- **Protocol Control Information (PCI)**: insieme di informazioni di protocollo contenute nell'header di un livello, costituisce l'overhead introdotto dal livello
- **Service Data Unit (SDU)**: insieme di dati che viene passato al livello inferiore, è composta dal payload e dall'header del livello superiore, e diventa il payload o SDU del livello inferiore.

Definizioni matematiche di throughput e goodput

Si considera $b_n(t)$ come il numero di bit trasmessi nell'intervallo di tempo $[0, t]$ dal livello n di rete al livello inferiore, e si definiscono le seguenti metriche di throughput e goodput:

$$S_n(t; T) = \frac{b_n(t) - b_n(t - T)}{T} \quad \text{throughput medio nell'intervallo di tempo } [t, t + T]$$

$$S_n(t) = \frac{db_n(t)}{dt} \quad \text{throughput istantaneo al tempo } t$$

$$S_n = \lim_{t, T \rightarrow \infty} S_n(t; T) \quad \text{goodput o throughput a lungo termine}$$

Si definiscono inoltre le metriche di throughput e l'efficienza in funzione della taglia delle PDU e SDU:

$$S_{n+1} = S_n \cdot \frac{|SDU_n|}{|PDU_n|} = S_n \cdot \frac{|SDU_n|}{|PCI_n| + |SDU_n|} \quad \begin{array}{l} \text{throughput massimo del} \\ \text{livello } n + 1 \text{ in funzione} \\ \text{di quello del livello } n \end{array}$$

$$\eta_n = \frac{S_{n+1}}{S_n} = \frac{|SDU_n|}{|PDU_n|} = \frac{|SDU_n|}{|PCI_n| + |SDU_n|} = \frac{1}{1 + \frac{|PCI_n|}{|SDU_n|}} = \frac{|PDU_{n+1}|}{|PDU_n|} \quad \begin{array}{l} \text{efficienza del livello } n \\ \text{con } 0 \leq \eta_n \leq 1 \end{array}$$

L'efficienza di un livello n indica la porzione di throughput del livello n che viene effettivamente utilizzata per trasmettere i dati utili del livello $n + 1$.

Bitrate di una connessione multi-hop

In caso di connessioni multi-hop (con più nodi intermedi), il bitrate complessivo è dato dal bitrate del collegamento più lento. Se si trasmette a bitrate superiori, si verificano fenomeni di congestione e perdita di pacchetti in quanto il collegamento più lento non riesce a gestire il flusso di pacchetti in arrivo.

8.4 Metriche di ritardo

Componenti macroscopiche del ritardo della rete

Le metriche di ritardo misurano il tempo impiegato dai dati per viaggiare da un nodo sorgente a un nodo destinazione attraverso la rete. Di seguito i principali tipi di ritardo a livello macroscopico di rete:

- **ritardo punto-punto** d_{p2p} : tempo che intercorre tra l'inizio della trasmissione di un pacchetto in un nodo e la ricezione completa dello stesso pacchetto in un nodo adiacente, a volte è indicato anche come ritardo nodale d_{nodal}
- **ritardo end-to-end** d_{e2e} : tempo che intercorre tra l'inizio della trasmissione di un pacchetto in un nodo e la ricezione completa dello stesso pacchetto nel nodo destinazione, è il risultato della somma di tutti i ritardi nodali lungo il percorso del pacchetto attraverso la rete

$$d_{e2e} = \sum_{i=1}^K d_{nodal, i} \quad d_{e2e} = K \cdot d_{nodal} \text{ se i link sono identici}$$

- **round trip time** d_{RTT} : tempo che intercorre tra l'inizio della trasmissione di un pacchetto in un nodo e la ricezione completa della risposta al pacchetto nello stesso nodo, è il risultato della somma dei ritardi end-to-end della trasmissione del pacchetto di richiesta e della risposta

$$d_{RTT} = 2 \cdot d_{e2e}$$

Componenti del ritardo nodale

Il ritardo nodale d_{nodal} si può scomporre nelle seguenti componenti:

$$d_{nodal} = d_{proc} + d_{queue} + d_{trans} + d_{prop}$$

- **ritardo di processing** d_{proc} : tempo necessario al nodo per elaborare il pacchetto, può essere dovuto a error checking, routing, ecc.; in genere è trascurabile siccome l'elaborazione è eseguita in hardware dedicati come ASIC, SmartNIC e DPU dedicate

- **ritardo di accodamento** d_{queue} : tempo che il pacchetto trascorre nel buffer in attesa di essere trasmesso sul link, dipende dal traffico di rete e può essere molto variabile; può essere parzialmente ridotto con l'uso di sistemi di scheduling, priorità e controllo di ammissione dei pacchetti
- **ritardo di trasmissione** d_{trans} : tempo che intercorre tra l'invio del primo bit e l'invio dell'ultimo bit del pacchetto sul link, equivale al rapporto tra la dimensione del pacchetto e il bitrate del link; può essere ridotto aumentando il bitrate del link; è detto anche tempo di forwarding o di inoltramento

$$d_{trans} = \frac{L}{R} = \frac{|PDU_{PHY}|}{S_{PHY}} \quad \text{con } L \text{ dimensione del pacchetto e } R \text{ bitrate del link}$$

- **ritardo di propagazione** d_{prop} : tempo necessario al pacchetto per propagarsi lungo il link tra due nodi adiacenti, equivale al rapporto tra la distanza tra i due nodi e la velocità di propagazione del segnale sul mezzo fisico; solitamente è trascurabile tranne in caso di link a lunga distanza come collegamenti satellitari o transoceanici; non può essere ridotto siccome non si può controllare la velocità di propagazione del segnale, al limite si può ridurre la distanza tra i due nodi

$$d_{prop} = \frac{d}{v} \quad \text{con } d \text{ distanza tra i due nodi e } v \text{ velocità di propagazione}$$

Jitter

Il jitter è la varianza del ritardo di trasmissione. Risulta critico per applicazioni in cui si hanno vincoli di latenza stringenti (es. real-time come videoconferenze, gaming online, telemedicina, ecc.) oppure per applicazioni in cui si richiede un flusso continuo e costante di dati (es. streaming video o audio). Il jitter può essere mitigato con l'uso di un buffer a discapito di una maggiore latenza complessiva.

8.5 Prodotto banda-ritardo (BDP) e ritrasmissione dei pacchetti

Prodotto banda-ritardo

Il prodotto banda-ritardo, o Bandwidth-Delay Product (BDP), indica il numero di bit o pacchetti che possono essere simultaneamente in transito attraverso una connessione di rete tra due nodi. Si calcola come il prodotto tra il throughput a regime stazionario S per il ritardo RTT d_{RTT} (invio del pacchetto e acknowledgment della ricezione):

$$BDP_{bit} = S \cdot d_{RTT} \quad BDP_{pkt} = \frac{BDP_{bit}}{|PDU_{PHY}|}$$

In caso di connessioni multi-hop, il BDP complessivo è calcolato in funzione del bitrate complessivo della connessione, ovvero quello del collegamento più lento. Il ritardo RTT risulta essere la somma dei ritardi RTT di tutti i collegamenti della connessione.

Ritrasmissione dei pacchetti

Per assicurarsi che un pacchetto sia stato correttamente ricevuto dal nodo destinatario, il nodo sorgente deve attendere un acknowledgment (ACK) di conferma. Se l'ACK non arriva entro un certo tempo di timeout, il pacchetto viene considerato perso o danneggiato ed eventualmente dovrà essere ritrasmesso. Per questo motivo serve mantenere un buffer di trasmissione per salvare i pacchetti inviati in attesa di ricevere l'ACK, per averli pronti per un'eventuale ritrasmissione.

La dimensione del buffer di trasmissione è pari al numero di pacchetti che possono essere simultaneamente in transito e di cui si deve attendere l'ACK, ovvero esattamente il prodotto banda-ritardo.

In caso di connessioni multi-hop, l'ACK viene inviato solo dal nodo destinatario finale, quindi il buffer di trasmissione deve garantire la memorizzazione di tutti i pacchetti in transito lungo l'intero percorso della connessione.

Prodotto banda-ritardo per Long Fat Networks (LFT) e connessioni multi-hop

Il prodotto banda-ritardo è particolarmente importante per le connessioni su Long Fat Networks (LFN), ovvero reti con elevata larghezza di banda e grande latenza come backbone transcontinentali, connessioni tra datacenter, connessioni satellitari e edge cloud.

In questi casi la latenza risulta essere molto elevata, di conseguenza anche il BDP risulta molto elevato e si richiede un buffer di trasmissione molto grande. Il problema è che le dimensioni del buffer sono limitate dal protocollo di trasporto. Se la dimensione massima non è sufficiente (è minore del BDP), non è possibile inviare altri pacchetti senza prima aver ricevuto l'ACK per quelli contenuti nel buffer, causando un calo del throughput della connessione.

8.6 Metriche di affidabilità

Packet error rate - PER

Il packet error rate (*PER*) equivale al rapporto in percentuale tra il numero di pacchetti persi o danneggiati e il numero totale di pacchetti trasmessi. Indica la probabilità che un pacchetto venga perso o danneggiato durante la trasmissione.

Dalla probabilità d'errore sul bit al packet error rate

Un canale di trasmissione punto-punto per bit viene modellato con un canale binario simmetrico (BSC) con probabilità d'errore ε sul bit. Si assume che gli errori sul bit siano indipendenti e identicamente distribuiti. È possibile passare dalla probabilità d'errore sul bit ε al packet error rate *PER* sul pacchetto di dimensione L bit con la seguente formula:

$$PER = 1 - (1 - \varepsilon)^L \quad \text{con } (1 - \varepsilon)^L = \begin{array}{l} \text{probabilità di ricevere tutti gli } L \text{ bit} \\ \text{che formano il pacchetto senza errori} \end{array}$$

È utile definire la probabilità $P(l)$ di ricevere l bit errati su L bit trasmessi:

$$P(l) = \binom{L}{l} \varepsilon^l (1 - \varepsilon)^{L-l} \quad \text{con } \binom{L}{l} = \# \text{ modi diversi di scegliere i bit errati}$$

Codifica di canale per rilevamento e correzione degli errori

Per poter rilevare e correggere gli errori sui bit introdotti dal canale, si aggiungono dei bit di ridondanza secondo le tecniche di codifica di canale. Esistono due tipi di codifica di canale:

- **codici a blocco:** si partiziona il flusso di bit in blocchi di k bit a cui si aggiungono r bit di ridondanza ottenendo blocchi in uscita di $n = k + r$ bit che vengono trasmessi sul canale; si definisce il tasso di codifica $R = k/n < 1$ come fattore di riduzione del throughput dovuto all'overhead dei bit di ridondanza
- **codici convoluzionali:** lavorano con lo stesso principio, ma l'uscita ad un certo istante di tempo dipende anche dai dati trasmessi agli istanti di tempo precedenti (è dotato di memoria)

Attualmente si utilizzano le seguenti codifiche di canale:

- **LDPC** o Low-Density Parity-Check: codici a blocco sparso con decodifica iterativa, utilizzati per dati utente trasmessi ad alta velocità (5G, Wi-Fi 7/8, Ottica 400G/800G)
- **polar codes:** codici basati su canali sintetici e decodifica successive cancellation, utilizzati per canali di controllo e pacchetti brevi (5G, 6G)
- **Reed-Solomon:** codici algebrici a blocchi su campi finiti, utilizzati per correzione di errori burst su link ottici o storage
- **turbo codes:** codici convoluzionali concatenati in parallelo, utilizzati nei sistemi legacy e in alcuni uplink 4G/5G

Rilevamento di errori e controllo di parità

I codici a controllo di parità consistono nell'aggiungere un bit di parità p al blocco di k bit da trasmettere, ottenendo un blocco di $n = k + 1$ bit. Il bit di parità è calcolato come la somma modulo 2 dei k bit del blocco, ovvero $p = 0$ se il numero di bit a 1 è pari, $p = 1$ se il numero di bit a 1 è dispari.

Questa struttura permette di rilevare fino ad 1 errore sul blocco di n bit, ma non permette di correggerlo. Si definisce la capacità di rilevamento errori su un flusso continuo pari a $1/n$ che corrisponde anche all'overhead introdotto. È possibile aumentare la capacità di rilevamento errori aggiungendo più bit di parità, ma aumenterà di conseguenza anche l'overhead e quindi il calo del throughput.

Correzione di errori e codice di Hamming(n,k)

I codici di Hamming sono codici a blocco che permettono di rilevare e correggere errori sui bit. Il codice di Hamming(n,k) aggiunge $r = n - k$ bit di ridondanza ogni k bit di dati, ottenendo blocchi di n bit. Riesce a rilevare fino a 2 errori sul blocco e a correggere fino a 1 errore sul blocco. Un esempio comune è il codice di Hamming(7,4) che aggiunge 3 bit di ridondanza ogni 4 bit di dati, con blocchi di 7 bit.

Errori in burst e interleaving

Si parla di errori in burst quando si hanno più errori consecutivi su un blocco di dati. Sono tipici dei canali di trasmissione reali in cui si hanno fenomeni di fading, interferenze o rumore impulsivo che si protraggono nel tempo e di conseguenza coinvolgono più bit consecutivi.

Il problema degli errori in burst si verifica quando si utilizzano codici a blocco, in quanto tutti gli errori si concentrano su uno o pochi blocchi, rendendone impossibile il rilevamento e la correzione. Per risolvere questo problema si utilizza la tecnica di interleaving che consiste nel rimescolare i bit di più blocchi in modo da distribuire gli errori e diminuire il numero di errori locale di ciascun blocco.

Ad esempio è possibile comporre una matrice di x righe composte dagli n bit di x blocchi consecutivi, e trasmettere i bit leggendoli per colonna. Lo svantaggio è che si introduce un ritardo di trasmissione in quanto per decodificare il primo blocco è necessario attendere la ricezione di tutti gli x blocchi.

Compromessi della codifica di canale

La codifica di canale comporta alcuni svantaggi:

- calo del bitrate pari al tasso di codifica R a causa dell'overhead introdotto dai bit di ridondanza
- aumento della complessità computazionale per l'encoding e il decoding dei blocchi di dati, maggiore è la capacità di rilevamento e correzione errori, maggiore è la complessità computazionale
- aumento del ritardo di trasmissione a causa dell'interleaving e delle codifiche convoluzionali

Congestione, packet loss rate e packet delivery ratio

Prima di essere inoltrati su un canale, i pacchetti vengono memorizzati in un buffer di trasmissione. Se il buffer si riempie a causa di un eccessivo traffico di rete, non è più possibile memorizzare i pacchetti in arrivo e si verifica un buffer overflow. I pacchetti in arrivo devono venire scartati secondo una delle possibili politiche di gestione del buffer come le seguenti:

- drop tail policy: se c'è un buffer overflow, si scartano tutti i pacchetti in arrivo
- early dropping casuale: quando l'occupazione del buffer supera una certa soglia (es. $2/3$), si scartano casualmente alcuni pacchetti in arrivo

In caso di perdita del pacchetto, questo può essere ritrasmesso dal nodo sorgente, dall'ultimo nodo intermedio o non essere ritrasmesso affatto, in base al protocollo di trasporto utilizzato.

Si definisce il packet loss rate ($PLOSS$) è definito come il rapporto in percentuale tra il numero di pacchetti persi e il numero totale di pacchetti trasmessi. Indica la probabilità che un pacchetto venga perso durante la trasmissione. Spesso viene utilizzato in modo intercambiabile con il packet error rate o packet drop rate (PER).

Come metrica complementare, si definisce anche il packet delivery ratio (PDR) come il rapporto in percentuale tra il numero di pacchetti ricevuti correttamente e il numero totale di pacchetti trasmessi e si calcola come $PDR = 1 - PLOSS$.

Si nota che le metriche di affidabilità PER , $PLOSS$ e PDR dipendono fortemente dal livello di rete in cui vengono misurate e dai protocolli implementati, in quanto dipende da come vengono gestiti il rilevamento, la correzione d'errori e la ritrasmissione dei pacchetti.

9 Streaming

9.1 Protocolli real-time (push) e streaming (pull)

Esistono diverse classi di applicazioni con requisiti differenti che devono operare tutti sulla stessa infrastruttura di rete IP già esistente. È necessario sviluppare protocolli di trasmissione appositi per soddisfare i requisiti di ciascuna classe sacrificando alcune funzionalità non necessarie.

Protocollo Real Time - push

Il protocollo real time o push è un protocollo leggero ed essenziale pensato per la trasmissione dati di comunicazioni real-time con requisiti di latenza molto stringenti. È usato per servizi di videoconferenza, cloud gaming e chirurgia remota. È composto da:

- **UDP** o User Datagram Protocol: protocollo di trasporto che offre solo un servizio di consegna dei pacchetti senza garanzia di consegna, è estremamente leggero e veloce, ma molto essenziale
- **RTP** o Real-Time Transport Protocol: aggiunge ai pacchetti UDP un header con informazioni di sequenza e timestamp per sincronizzazione, ordinamento e gestione del jitter
- **WebRTC** o Web Real-Time Communication: framework di sistema, aggiunge cifratura dei dati, meccanismi per attraversare NAT e firewall, accesso hardware per gestire microfono e webcam direttamente dal browser

Protocollo Streaming - pull

Il protocollo streaming o pull è un protocollo più complesso e pesante pensato per la trasmissione dati di contenuti multimediali che richiedono maggiore affidabilità della trasmissione, ma tollerano una latenza maggiore. È composto da:

- **CDN** o Content Delivery Network: rete di server distribuiti geograficamente che memorizzano copie dei media da trasmettere, permettono di ridurre la latenza siccome avvicinano i contenuti agli utenti finali, riducono il traffico sulla rete principale siccome la distribuzione avviene tramite reti locali e aumentano l'affidabilità grazie alla ridondanza dei contenuti
- **QUIC/TCP** o Quick UDP Internet Connections/Transmission Control Protocol: protocolli di trasporto che offrono un servizio di trasmissione a pacchetti affidabile, garantendone la consegna
- **HTTP/3** o Hypertext Transfer Protocol: protocollo di livello applicativo che permette la trasmissione di contenuti multimediali tramite il protocollo di trasporto QUIC/TCP
- **MPEG-DASH/HLS** o Dynamic Adaptive Streaming over HTTP/HTTP Live Streaming: protocolli di streaming che gestiscono la trasmissione dei contenuti multimediali tramite il protocollo HTTP/3, adattando la qualità in base alle prestazioni della connessione dell'utente finale
- **codec video**: algoritmi di compressione e decompressione che avvengono all'interno dell'applicativo nella macchina client

9.2 Adattamento del tasso di codifica video

Scelta del tasso di codifica video

In generale il tasso di codifica video deve essere inferiore al throughput della rete $R_C \leq S$. Ad oggi si conoscono strategie avanzate per adattare il tasso di codifica video in tempo reale alle prestazioni della rete, ma spesso le prestazioni della rete non si conoscono a priori, specialmente se le fasi di codifica e di trasmissione avvengono in momenti diversi o in contesti di distribuzione in cui un unico server offre servizi a molti client.

Se il tasso di codifica è molto più basso rispetto al throughput disponibile, la qualità video sarà molto bassa e l'esperienza utente sarà scadente. Se il tasso di codifica è più alto rispetto al throughput disponibile, si verificheranno fenomeni di buffer overflow e di perdita dei pacchetti, causando interruzioni nella riproduzione del video e un'esperienza utente scadente.

Di seguito si analizzano alcune strategie per garantire la massima qualità video possibile limitatamente alle prestazioni della rete, gestendo adeguatamente il tasso di codifica video.

Compressione estrema

L'approccio più semplice è quello di comprimere il video al massimo, riducendo il bitrate al minimo possibile. In questo modo si garantisce un bitrate inferiore al throughput della rete, ma la qualità video sarà molto bassa anche quando la rete permette la trasmissione di video con qualità maggiore a bitrate più elevati. L'esperienza utente sarà quindi scadente.

Transcodifica

L'approccio prevede che le macchine router o server effettuino una transcodifica del video in tempo reale per abbassare la qualità in caso di congestione della rete. Questo approccio ha alcune criticità fondamentali:

- il costo computazionale è insostenibile per le macchine router o server
- non è possibile accedere ai dati trasmessi in quanto cifrati end-to-end
- un banale packet drop sistematico non può funzionare in quanto gli algoritmi sono altamente predittivi e poco tolleranti ad errori sui frame

Scalable video coding - SVC

L'approccio consiste nell'utilizzare dal lato server una codifica video scalabile, ovvero composta da più flussi video di qualità sempre crescente. Si codifica inizialmente un video in un flusso base con alti tassi di compressione e quindi bassa qualità video e basso bitrate, in modo da garantire continuità su reti con throughput limitato. Successivamente si codificano altri flussi detti enhancement che aggiungono progressivamente sempre più informazioni di perfezionamento del flusso base per aumentare frame rate, risoluzione e qualità video.

Il server trasmette contemporaneamente sia il flusso base che i flussi enhancement. Se si dovessero verificare fenomeni di congestione della rete, il server o i nodi intermedi possono decidere di interrompere la trasmissione di uno o più flussi enhancement. In questo modo si abbassa il bitrate e si mantiene la continuità di trasmissione del flusso base per garantire fluidità del video.

Adaptive bitrate - ABR

La soluzione ad oggi più adottata è quella del client-driven adaptive bitrate (ABR). Consiste in un protocollo di "collaborazione" a livello applicativo tra client e server senza richiedere requisiti all'infrastruttura di rete e ai livelli inferiori.

In pratica il server suddivide il video in segmenti di durata fissa, tipicamente 2-10 secondi, e li codifica in più varianti con bitrate e qualità differenti e li offre al client. L'applicativo dal lato client si occupa di decidere quale versione scaricare in base alle prestazioni della rete e alla qualità video desiderata.

Per permettere al client di effettuare la scelta del segmento migliore da scaricare, il server mette a disposizione un file detto Media Description Presentation (MDP) con le informazioni necessarie, come:

- numero di segmenti disponibili e durata di ciascun segmento
- numero di livelli di qualità disponibili per ciascun segmento
- bitrate, risoluzione e frame rate di ciascun livello
- l'url di ciascun segmento per ciascun livello

9.3 Dynamic adaptive streaming over HTTP - DASH

Descrizione del protocollo

L'implementazione più diffusa dell'approccio ABR è il protocollo DASH, definito nello standard MPEG-DASH. Il protocollo descrive un modello pull, dove il client inizialmente richiede al server il file MDP e progressivamente scarica dal server i segmenti video che ritiene migliori uno alla volta. La scelta del segmento viene fatta lato client. I dati viaggiano su protocolli di trasporto HTTP/3 e QUIC, secondo il modello pull (il client richiede i segmenti al server).

Vantaggi architetturali di HTTP/3 e QUIC rispetto a RTSP/RTMP e TCP

L'utilizzo di HTTP/3 e QUIC (basati su UDP) permette di superare le limitazioni dello streaming su protocolli legacy RTSP/RTMP e del trasporto basato su TCP, introducendo i seguenti miglioramenti:

- **approccio web-centrico**: si abbandona il modello push e stateful su porte non standard web dei formati RTSP/RTMP, in favore di un'architettura pull e stateless in cui la gestione del traffico è affidata al client; l'uso di porte standard (80/443) garantisce l'attraversamento trasparente di NAT e firewall, e permette la simbiosi con le CDN per il caching
- **assenza di HOL blocking**: il multiplexing nativo di UDP permette di avere più flussi indipendenti e la perdita di un pacchetto non blocca l'intera coda di trasmissione come nel TCP
- **assenza di latenza di Setup**: vengono eliminati i multipli handshake sequenziali imposti da TCP, consentendo l'avvio quasi istantaneo della trasmissione dati
- **connection migration**: la connessione non è vincolata alla coppia IP/Porta, garantendo la continuità dello streaming anche durante i cambi di rete (es. transizione da Wi-Fi a rete mobile).
- **evoluzione rapida**: lo spostamento della logica di controllo della congestione dal kernel allo strato applicativo facilita i futuri aggiornamenti del protocollo

Stato di salute del buffer e gestione del rebuffering

Si definisce lo stato di salute del buffer del client $B(t)$ come la quantità di secondi di video presenti nel buffer all'istante t . Quando $B(t) = 0$ il buffer è vuoto ed è necessario effettuare un rebuffering, ovvero interrompere la riproduzione per riempire il buffer con un certo numero (L o M) di nuovi segmenti.

- L : # segmenti da scaricare durante il primo rebuffering, prima di iniziare la riproduzione
- M : # segmenti da scaricare durante un rebuffering intermedio, prima di riprendere la riproduzione

I parametri M e L vengono scelti dal servizio di streaming. Più grandi sono, maggiore è la durata del rebuffering, ma minore sarà la probabilità di un nuovo rebuffering successivo.

In condizione di playout, ovvero durante la riproduzione del video, lo stato del buffer $B(t)$ decresce al tasso di un secondo al secondo a cui viene sommata la quantità di video scaricato. In condizioni di rebuffering, invece, lo stato del buffer $B(t)$ aumenta al tasso di video scaricato.

$$\frac{dB_{\text{playout}}(t)}{dt} = \frac{S}{R_C} - 1 \quad \text{con} \quad \begin{array}{l} S < R_C \rightarrow \text{buffer in svuotamento} \\ S > R_C \rightarrow \text{buffer in riempimento} \end{array} \quad \Bigg| \quad \frac{dB_{\text{rebuffering}}(t)}{dt} = \frac{S}{R_C}$$

Tempo di download di un segmento

Si definisce il tempo di download $T_D(n, k)$ del segmento n al livello di qualità k come rapporto tra il tempo di download (prodotto tra la durata del segmento in secondi T_S e il tasso di codifica $R_C(k)$) e il throughput della rete S_n durante il download del segmento n :

$$T_D(n, k) = \frac{T_S \cdot R_C(k)}{S_n} \implies \frac{T_D(n, k)}{T_S} = \frac{R_C(k)}{S_n}$$

Quality of Experience - QoE

La Quality of Experience (QoE) è una metrica che misura la qualità percepita dall'utente finale durante la fruizione di un contenuto multimediale ed è data dalla seguente formula:

$$J = \underbrace{\sum_{n=1}^N q(n)}_{\text{qualità del video}} - \underbrace{\lambda \cdot \sum_{n=1}^N |q(n) - q(n-1)|}_{\text{penalizzazione per cambio di qualità}} - \underbrace{\mu \cdot \sum_{n=1}^N T_{\text{stall}}(n)}_{\text{penalizzazione per gli stalli}}$$

- $q(n)$: qualità del segmento n in base al livello di codifica scelto
- λ : peso della penalizzazione per il cambio di qualità tra due segmenti consecutivi
- μ : peso della penalizzazione per gli stalli dovuti al rebuffering (solitamente molto alto)